Quality Shapes Extraction from very large Knowledge Graphs

Kashif Rabbani
Aalborg University, Denmark
kashifrabbani@cs.aau.dk

Matteo Lissandrini
Aalborg University, Denmark
matteo@cs.aau.dk

Katja Hose
Aalborg University, Denmark
khose@cs.aau.dk

ABSTRACT

Knowledge Graphs (KGs) represent heterogeneous domain knowledge on the Web and within organizations. There exists shapes constraint languages to define *validating shapes* for ensuring the quality of the data in KGs. Existing techniques to extract validating shapes often fail to extract complete shapes, are not scalable, and are prone to produce spurious shapes. To address these shortcomings, we propose a scalable Quality Shapes Extraction (QSE) approach to extract validating shapes in very large graphs, and we devise both an exact and an approximate solution. QSE provides information about the reliability of shape constraints by computing their confidence and support within a KG and in doing so allows to identify those shapes that are most informative and less likely to be affected by incomplete or incorrect data. We are the first to extract a complete set of validating shapes from WikiData and we demonstrate the superior scalability of QSE which provides a 12x reduction in extraction time. Additionally, QSE is able to filter out up to 93% of the invalid and spurious shapes resulting in a reduction of up to 2 orders of magnitude in the number of constraints presented to the user, e.g., from 11,916 to 809 on DBpedia.

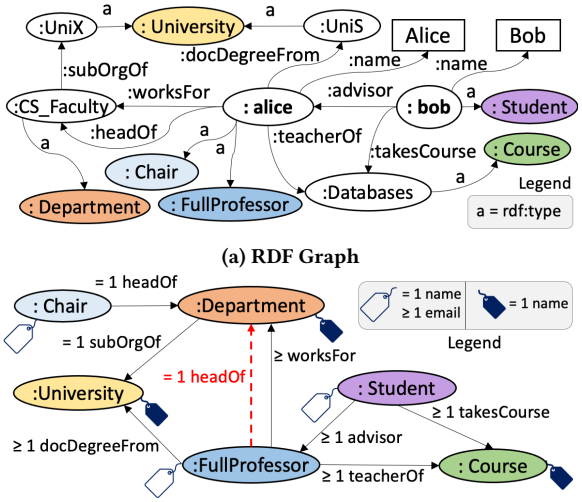
PVLDB Reference Format:
Kashif Rabbani, Matteo Lissandrini, and Katja Hose. Quality Shapes Extraction from very large Knowledge Graphs. PVLDB, 16(1): XX-XX, 2023. doi:X.X/XX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/kashif-rabbani/shacl.

1 INTRODUCTION

Knowledge Graphs (KGs) are in widespread use both within companies [44, 45] and on the Web [50, 53]. They represent entities and their relationships in various domains [31, 45] as collections of $\langle subject, relation, object \rangle$ triples using the Resource Description Framework (RDF) [8]. KGs are used in various data-centric AI tasks such as search engines, question answering, and smart assistants [17]. Nonetheless, as more and more data is accrued within KGs, practical applications impose further demands, especially in terms of quality assessment and validation [41].

Therefore, Shapes constraint languages (e.g., SHACL [24], and ShEx [36]) have been proposed to validate KGs by enforcing constraints represented in the form of *validating shapes*. For instance, both specifications can express that an entity of type Student needs to have a name, a registration number, and should be enrolled to some courses; and that these attributes should be instances of type string, date, and Course, respectively – see Figure 1a for an example KG and Figure 1b for a simplified depiction of its corresponding set of validating SHACL shapes.



(a) RDF Graph

(b) Validating Shapes

Figure 1: An example RDF Graph and Validating Shapes

Often, validating shapes are manually specified by domain experts. Yet, when trying to specify validating shapes for already-existing large-scale KGs, data scientists are in need of tools that can speed-up this process [41]. Thus, various tools have been proposed to produce automatically [7, 11, 20, 28] or semi-automatically [4, 34, 38] a set of validating shapes for a target KG. Unfortunately, these methods suffer of 3 important limitations: (1) they are not able to produce complete shapes, e.g., they can identify that a student should have a property of type takesCourse but they do not extract the fact that the object should be of type Course; (2) the shapes they produce are easily affected by errors and inconsistencies in the KG, e.g., if one single department, by mistake, is attached the property hasAdvisor, a corresponding spurious shape is also extracted; and (3) they have not been designed to scale to large-scale KGs, i.e., they cannot process at all the full english WikiData KG and they take days to process a subset of it. *Therefore in this work we present the first techniques for efficient extraction of validating shapes from very large existing KGs that also ensure robust methods to limit the effect of spuriousness.*

We found the effect of spuriousness to become particularly , in DBpedia [3], some of the musical bands entities are wrongly assigned to the class dbpedia.org/ontology/City. As a consequence, when shapes are extracted from its instance data using the existing approaches, the resulting node shape for dbo:City contains node and property shapes specifying cities to have properties like

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 16, No. 1 ISSN 2150-8097.
doi:X.X/XX.XX

dbo:genre and dbo:formerBandMember. Since existing KGs are very rich in terms of node and edge types, and due to the effect of *spuriousness*, existing approaches generate tens of thousands of these shapes (our experiments show that more than 82K node shapes and 2 million property shapes are generated by a trivial extraction process for WikiData [53]), thus it becomes unmanageable for domain experts to manually curate the shapes to establish their validity and usefulness. The only existing approach that attempted to tackle this issue is SheXer [11], which supports filtering of shapes based on a "trustworthiness" score. Unfortunately, this score does not directly translate into how frequently a shape is satisfied in a dataset, so it is still prone to generate spurious shapes and it is also hard to tune. Furthermore, it is not designed to handle very large KGs, and requires more than 2 days to extract shapes from a smaller sample of WikiData.

Given *scalability* and *spuriousness* as two common limitations in state-of-the-art, we introduce the problem of *support-based shapes extraction* and propose QSE (Quality Shapes Extraction) as a solution to overcome these limitations. We devise both, QSE-Exact and QSE-Approximate solutions. QSE-Exact approach further has two variants, i.e., a file-based version which saves us the time and effort required to load the data into a triplestore, and a query-based version which allows to extract shapes from the data already loaded into a triplestore. Both versions work independent from each other. QSE-Exact (file-based) approach for shape extraction is independent of querying time (as it only deals with files), however, shape extraction in query-based approach relies on the performance of the triplestore. We focus on optimizing file-based shape extraction approach in this paper. To do so, we propose efficient algorithms for *constraints extraction*, *computation of support and confidence*, *construction of SHACL shapes* with *quality guarantees* from very large graph files. Support and confidence represents the number and ratio of the nodes and properties in the graph conforming to a given shape. It allows a user to specify a minimum threshold for confidence and support for the constraints that are going to be extracted. *Hence, QSE can filter out shapes affected by spurious or erroneous data.* Additionally, to enable shape extraction on a commodity machine, we propose an efficient approximation algorithm (QSE-Approximate) to perform sampling of entities of very large KGs based on dynamic neighbor-based reservoir sampling.

We perform a thorough experimental evaluation using both synthetic (LUBM [15]) and real (DBpedia [3], YAGO-4 [50], WikiData [53])) KGs demonstrating the benefits of our approach. The results show that our QSE-Exact approach can extract shapes from WikiData [53])'s 2015 dump in 16 minutes and 2021's dump in 2.56 hours. Similarly, our QSE-Approximate approach is able to extract shapes from WikiData [53]'s 2021 dump in 90 minutes by outputting node shapes with 100% precision and recall and property shapes with 100% precision and 95% recall.

2 MOTIVATING EXAMPLE

We present a subset of LUBM [15] knowledge graph in Figure 1a. It represents a student named Bob who takes a database course taught by Alice, a Full Professor working as the head of the Computer Science department at UniX. Additionally, Alice has received their doctoral degree from UniS. Analogously, Figure 1b depicts a set of validating shapes for the same KG. In particular, Figure 1b represents *Node Shapes* and *Property Shapes* each annotated with a set of constraints. For example, the node shape Student declares the constraints that all students must have one name and take at least one course. Moreover, the edge between the node shape Student and the node shape Course represents the property shape takesCourse with its cardinality constraint. We provide a syntactical example of shapes in the extended version (appendix A.1) of our paper.

Existing approaches to generate validating shapes for a given RDF graph will iterate over existing triples, e.g., (:alice, :teacherOf, :Databases), or (:alice, :name, "Alice") , obtain the type of the subject and the object in the triple, and then produce the corresponding node and property shapes. Unfortunately, this approach will produce spurious shapes. For instance, since :alice has both type Full Professor and Chair, when parsing the triple (:alice, :headOf, :CS_Faculty), the property shape headOf (the red dotted arrow in Figure 1b) is assigned to both node shapes, instead of assigning it to the Chair node shape only. To avoid this, we could compare within the entire graph how often nodes that are instances of Chair have the :headOf compared instead to nodes that are instances of Full Professor only. Additionally, shapes extracted by existing approaches are incomplete in the sense that, since they do not keep track of the frequency of property shapes, they simply adopt the open-world assumption and assign minCount=0 for all cardinality constraints. Therefore, we propose instead to explicitly keep track of the frequency of node and property shape occurrences to reduce the number of spurious shapes produced and produce instead complete shapes annotated with information regarding their frequency to allow domain experts to evaluate the reliability of the constraints that have been produced.

3 PRELIMINARIES AND PROBLEM

In this section, we begin by formally introducing preliminaries, formulating the problem statement, and finally explaining our proposed solution as Quality Shapes Extraction (QSE) approach.

3.1 Preliminaries

Here we briefly define KGs followed by the syntax and semantics of validating shapes.

3.1.1 Knowledge Graphs. The standard format for encoding knowledge graphs is the Resource Description Framework (RDF [8]). It is a W3C standard format to publish and exchange data over the Web as a finite set of $\langle s, p, o\rangle$ triples stating that a subject s is in a relationship with an object o through predicate p. Therefore, we define an RDF graph (also shown in Figure 1a) as follows:

Definition 3.1 (RDF graph). Given pairwise disjoint sets of IRIs $\mathcal{I}$, blank nodes $\mathcal{B}$, and literals $\mathcal{L}$, an RDF Graph $\mathcal{G}$ is a finite set of RDF triples $\langle s, p, o\rangle \in (\mathcal{I} \cup \mathcal{B}) \times \mathcal{I} \times (\mathcal{I} \cup \mathcal{B} \cup \mathcal{L})$.

Moreover, we distinguish two special subsets of the IRIs $\mathcal{I}$: predicates and classes. The set of predicates $\mathcal{P} \subset \mathcal{I}$ is the subset of IRIs that appear in the predicate position p in any $\langle s, p, o\rangle \in \mathcal{G}$. Among predicates $\mathcal{P}$, we identify the type predicate $\mathbf{a} \in \mathcal{P}$, which corresponds to IRI rdf:type [55] or wdt:P31 in WikiData [53] as instance of property. When a type predicate is used in the triple as $\langle s, \mathbf{a}, o\rangle$, it states that the entity appearing as subject s is an instance-of the

class (also called type) represented by the object o. Therefore, we also identify the subset $C : \{c \in \mathcal{I} | \exists s \in \mathcal{I}$ s.t. $\langle s, \mathbf{a}, c \rangle \in \mathcal{G}\} \subset \mathcal{I}$ of all the IRIs that are classes in $\mathcal{G}$.

3.1.2 Validating Shapes. A set of *validating shapes* represents integrity constraints in the form of a shape schema over RDF graphs distinguished by two components, i.e., the *data graph* and the *shape graph*. The *data graph* is the RDF graph $\mathcal{G}$ to be validated, while the *shape graph* consists of constraints in the form of the shape schema $\mathcal{S}$ against which entities of the data graph are validated. These constraints are defined using node and property shapes. In the following, we define the syntax of $\mathcal{S}$ according to SHACL *core constraint components* [54] by adopting the syntax introduced by Ognjen et al. [43].

Definition 3.2 (SHACL Syntax). A SHACL Shape schema $\mathcal{S}$ consists of a set of node shapes defined as $\langle s, \tau_s, \Phi_s \rangle$, where s is the shape *name*, $\tau_s \in C$ is the *target definition*, and Φ_s is a set of property constraints of the form $\phi_s{:}\langle \tau_\mathsf{p}, \mathsf{T_p}, \mathsf{C_p} \rangle$, where $\tau_\mathsf{p} \in \mathcal{P}$ is called the target property, $\mathsf{T_p} \subset \mathcal{I}$ contains either an IRI defining a *literal type*, e.g., xsd:string, or a set of IRIs – called *class type constraint*, and $\mathsf{C_p}$ is a pair $(n, m) \in \mathbb{N} \times (\mathbb{N} \cup \{\infty\})$ – called *cardinality constraints*.

A node shape is defined for the instances of a *target class* $\tau_s \in C$ and lists a set of property shapes Φ_s defining which properties each instance of τ_s can or should be associated with. In Figure 1b, the student shape $\langle s, \tau_s, \Phi_s \rangle : \langle$ sh:StudentShape, :Student, $\{\phi_{s_1}, \phi_{s_2}\} \rangle$ contains a node shape for target class :Student having two property shapes ϕ_{s_1} and ϕ_{s_2} The property shape ϕ_{s_1} has a target property τ_p= sh:path :name, a literal type constraint $\mathsf{T_p}$= sh:datatype xsd:string, and cardinality constraints $\mathsf{C_p}$= { sh:minCount 1, sh:maxCount 1 }. Similarly, the property shape ϕ_{s_2} has a target property τ_p= sh:path :takesCourse, a class type constraint $\mathsf{T_p}$= sh:class :Course, and a cardinality constraint $\mathsf{C_p}$ equivalent to $(1, \infty)$.

When validating a graph $\mathcal{G}$ against a shape schema $\mathcal{S}$ having a node shape $\langle s, \tau_s, \Phi_s \rangle \in \mathcal{S}$, we say that an entity $e \in \mathcal{G}$ verifies s in $\mathcal{G}$ if the constraint Φ_s applied to e is valid in $\mathcal{G}$. We define the semantics of $\mathcal{S}$ as follows:

Definition 3.3 (SHACL Semantics). Given an entity e of a Graph $\mathcal{G}$, and a property shape $\phi_s{:}\langle \tau_\mathsf{p}, \mathsf{T_p}, \mathsf{C_p} \rangle$, we say that $e \models_\mathcal{G} \phi$, if the following conditions hold:

- If $\mathsf{T_p}$ is a literal type constraint, then for every triple $(e, \tau_p, l) \in \mathcal{G}$, l is a literal of type $\mathsf{T_p}$.
- If $\mathsf{T_p}$ is a set of class type constraints $\mathsf{T_p} = \{t_1, t_2, ...t_n\}$, then for every triple $(e, \tau_p, o) \in \mathcal{G}$, there exists a triple $(o,$ rdf:type /rdfs:subClassOf$^*, t) \in \mathcal{G}$ where $t \in \mathsf{T_p}$ and $e \models_\mathcal{G} \forall\, t \in \mathsf{T_p}$.
- $n \leq |\{(s, p, o) \in \mathcal{G} : s = e \wedge p = \tau_p\}| \leq m$, where $\mathsf{C_p}$=(n, m).

3.2 Shapes Support and Confidence

We introduce the notion of *support* and *confidence* for shape constraints to study the reliability of extracted shapes. These concepts are inspired by the well-known theory developed for the task of frequent patterns mining [16] and MNI support [6]. The MNI support is the minimum cardinality of *set of all nodes of $\mathcal{G}$ that are mapped to a specific pattern node by some isomorphism* across all the nodes of the pattern. In our approach, we consider support of a constraint in a shape as a cardinality of entities conforming to a specific constraint $\phi_s{:}\langle \tau_\mathsf{p}, \mathsf{T_p}, \mathsf{C_p} \rangle \in \Phi_s$ of a shape $\langle s, \tau_s, \Phi_s \rangle \in \mathcal{S}$ occurring in data graph $\mathcal{G}$. On the other hand, the confidence of a constraint ϕ_s measures the ratio between how many entities conform to ϕ_s and the total number of entities that are instances of the target class of s.

Definition 3.4 (Support of ϕ_s). Given a shape $\langle s, \tau_s, \Phi_s \rangle \in \mathcal{S}$ with shape constraint $\phi_s{:}\langle \tau_\mathsf{p}, \mathsf{T_p}, \mathsf{C_p} \rangle \in \Phi_s$, the support of ϕ_s is defined as the number of entities e satisfying ϕ_s, denoted as $e \models \phi_s$, hence:

$$\text{supp}(\phi_s) = |\{e \in \mathcal{I} | e \models \phi_s\}| \tag{1}$$

Definition 3.5 (Confidence of ϕ_s). Given a shape $\langle s, \tau_s, \Phi_s \rangle \in \mathcal{S}$ having shape constraint $\phi_s{:}\langle \tau_\mathsf{p}, \mathsf{T_p}, \mathsf{C_p} \rangle \in \Phi_s$, the confidence of ϕ_s is defined as the proportion of entities for which $e \models \phi_s$ among the entities that are instances of the target class τ_s of $s \in \mathcal{S}$, hence:

$$\text{conf}(\phi_s) = \frac{\text{supp}(\phi_s)}{|\{e | (e, type, \tau_s) \in \mathcal{G}\}|} \tag{2}$$

As it happens in the case of frequent pattern mining [16], when extracting validating shapes, the support provides insights on how frequently a constraint is matched in the graph, i.e., the number of entities e satisfying a constraint ϕ_s. While, similar to the task of itemset mining [5], the confidence can tell us how strong is the association between a node type and a specific constraint, i.e., the proportion of entities e satisfying a constraint ϕ_s among all the entities that are instances of the node type τ_s of $s \in \mathcal{S}$. For instance, the confidence for property shape headOf (Figure 1b) in our snapshot of LUBM is 10% for the Full Professor node shape and 100% for Chair, which indicates strong association of the headOf property shape to latter and a weak association to the former.

3.3 Problem Statement

Given the need to produce shapes limiting the effect of spuriousness in the graph, we formally define the problem of extracting high-quality shapes from KGs as follows:

Problem 1 (Quality Shapes Extraction). *Given an RDF graph $\mathcal{G}$, a threshold ω for support, and ε for confidence, the problem of quality shapes extraction over $\mathcal{G}$ is to find the set of shapes $\mathcal{S}$ such that for all node shapes $\langle s, \tau_s, \Phi_s \rangle \in \mathcal{S}$ $\text{supp}(s) > \omega$ whereas for all property shapes $\phi_s{:}\langle \tau_p, T_p, C_p \rangle \in \Phi_s$ $\text{supp}(\phi_s) > \omega$ and $\text{conf}(\phi_s) > \varepsilon$.*

In the following, we provide both, an exact and an approximate solution to the problem of quality shape extraction.

4 QSE-EXACT

In this section, we present our solution to Problem 1. Extracting shapes $\mathcal{S}$ from an RDF graph $\mathcal{G}$ requires processing of its triples and analysis of the types of nodes involved as subjects and objects in those triples. One of the most common ways to store an RDF graph $\mathcal{G}$ on a file **F** is to represent it as a sequence of triples. QSE reads **F** line by line and processes it as a stream of $\langle s, p, o \rangle$ triples. Algorithm 1 presents the four main steps of QSE to extract shapes from **F** of Graph $\mathcal{G}$.

In Entity extraction phase, the algorithm parses each $\langle s, p, o \rangle$ triple containing a type declaration; and for each entity it stores the set of its entity types and their frequency, i.e., the number of instances for each class (Lines 4-8) in maps Ψ_{ETD} (Entity to Data) and Ψ_{CEC} (Class to Entity Count), respectively. Once the types of all

Algorithm 1 SHAPES EXTRACTION

Input: Graph $\mathcal{G}$ from File **F**, ω: min-support, ε: min-confidence
Output: Output: $\mathcal{S}\langle s, \tau_s, \Phi_s\rangle$

1: $E_{data} \leftarrow \{$T: SET$_{\text{Types}}$, Ψ_{ETPD} = MAP$\langle$IRI, $P_{data}\rangle\}$
2: $P_{data} \leftarrow \{$T′ : SET$_{\text{ObjTypes}}$, Count :INT$\}$
3: Ψ_{ETD} = MAP$\langle$IRI, $E_{data}\rangle$, Ψ_{CEC} = MAP$\langle$IRI, INT$\rangle$, Ψ_{CTP} = MAP$\langle$IRI, MAP$\langle$IRI, SET$\rangle\rangle$
▷ ① Entity extraction
4: **for** $t \in \mathcal{G} \wedge$ t.p = Type Predicate **do**
5: entity e : t.s ; entityType e_t = t.o ▷ s: subject, o: object
6: **if** $e \notin \Psi_{\text{ETD}}$ **then** Ψ_{ETD}.insert(e, ...))
7: Ψ_{ETD}.insert(e, Ψ_{ETD}.get(e).T.add(e_t)) ▷ T : entity types
8: increment entity count for current e_t in Ψ_{CEC}
▷ ② Entity constraints extraction
9: **for** $t \in \mathcal{G} \wedge$ t.p != Type Predicate **do**
10: $\lambda_{\text{TYPE}(t.o)} \leftarrow \emptyset$, $\lambda_{\text{TUPLE}} \leftarrow \emptyset$ ▷ Initialize a *type* and *property to type* tuple set
11: **if** object $t.o$ is Literal **then**
12: $\lambda_{\text{TYPE}(t.o)}$.add(getLiteralType(t.o))
13: λ_{TUPLE}.add(new Tuple$\langle t.p$, getLiteralType$(t.o)\rangle\rangle$
14: **else** ▷ for non-literal objects
15: **for** $obj_{type} \in \Psi_{\text{ETD}}$.get($t.o$).T **do**
16: $\lambda_{\text{TYPE}(t.o)}$.add($obj_{type}$)
17: λ_{TUPLE}.add(new Tuple$\langle t.p, obj_{type}\rangle$)
18: updatePropertyConstraints($t.s$, λ_{TUPLE})
19: **for** IRI $\in \Psi_{\text{ETD}}$.get($t.s$.T) **do** ▷ if $t.s \in \Psi_{\text{ETD}}$
20: update Ψ_{CTP} with class IRI for property $t.p$
▷ ③ Support and Confidence computation
21: Ψ_{SUPP} = MAP$\langle$TUPLE$_3$, INT$\rangle$, Ψ_{CONF} = MAP$\langle$TUPLE$_3$, INT$\rangle$, Ψ_{PTT}
22: **for** $(e, E_{data}) \in \Psi_{\text{ETD}}$ **do**
23: **for** $(T, \Psi_{\text{ETPD}}) \in E_{data}$ **do**
24: **for** $e_t \in T \wedge (p, p_o, c) \in P_{data}$ **do**
25: $\chi \leftarrow$ createTriplets($\langle\tau_e, \tau_p, \tau_{po}\rangle$)
26: computeSupportAndConfidence (Ψ_{SUPP}, χ)
27: computeMaxCardinality (Ψ_{PTT}, p , c)
▷ ④ Shapes extraction
28: **for** (class, MAP$\langle$Property, SET$_{\text{ObjTypes}}\rangle$) $\in \Psi_{\text{CTP}}$ **do**
29: $\Phi_s \leftarrow \emptyset$ ▷ Property shapes $\Phi_s = \{\phi_{s_1}, \phi_{s_2}, ..., \phi_{s_n}\}$ where ϕ_s:$\langle\tau_{\text{p}}, \text{T}_{\text{p}}, \text{C}_{\text{p}}\rangle$
30: s = class.buildShapeName() , τ_s = class
31: **for** $(p$, SET$_{\text{ObjTypes}}) \in$ MAP$\langle$Property, SET$\rangle$ **do**
32: $\phi_s.\tau_{\text{p}} = p$
33: $p.\omega = \Psi_{\text{Supp}}$.get($p$, SET$_{\text{ObjTypes}}$) ▷ Support of property p
34: $p.\varepsilon = \Psi_{\text{Conf}}$.get($p$, SET$_{\text{ObjTypes}}$) ▷ Confidence of property p
35: **if** $p.\omega > \omega \wedge p.\varepsilon > \varepsilon$ **then**
36: **if** p is LITERAL **then** *for each* (objType $\in$ SET$_{\text{ObjTypes}}$)
37: **if** |SET$_{\text{ObjTypes}}$| > 1 **then** encapsulate all $\phi_s.\text{T}_{\text{p}}$ in sh:or
38: buildConstraints(sh:nodeKind, sh:datatype, sh:maxCount, Ψ_{PTT})
39: **else** *for each* (objType $\in$ SET$_{\text{ObjTypes}}$)
40: **if** |SET$_{\text{ObjTypes}}$| > 1 **then** encapsulate all $\phi_s.\text{T}_{\text{p}}$ in sh:or
41: buildConstraints(sh:nodeKind, sh:class, sh:maxCount, Ψ_{PTT})
42: $\phi_s.\text{C}_{\text{p}}.add$(sh:minCount : 1) ▷ $if p.\varepsilon > \varepsilon'$
43: $\Phi_s.add(\phi_s)$
44: $\mathcal{S}.add(s, \tau_s, \Phi_s)$ ▷ if s.$\omega > \omega \wedge \phi_s!\emptyset$

entities and their counts are computed, the algorithm enters into the second phase, i.e., Entity constraints extraction to perform a second pass over **F** (Lines 9-20) to collect the constraints and the meta-data required to compute support and confidence of each candidate shape. Specifically, it parses the triples to obtain subject and object types of all triples except for the type defined triples as they are already processed in the first phase. Here, the map Ψ_{ETD}, filled in the first phase, is used to identify the types of non-literal objects, while the map Ψ_{ETPD} (Entity to Property Data) is updated to contain the candidate property constraints associated with each entity (Lines 14-20).

In the third phase (i.e., Support and Confidence computation), the constraints' information stored in maps (Ψ_{ETD}, Ψ_{CEC}) is used to compute support and confidence for specific constraints. The algorithm iterates over the map Ψ_{ETD} to get the inner map Ψ_{ETPD} mapping entities to candidate property shapes ϕ_s:$\langle\tau_{\text{p}}, \text{T}_{\text{p}}, \text{C}_{\text{p}}\rangle \in \Phi_s$, and retrieves the type of each entity using types information stored in Ψ_{ETD} to build triplets for each distinct triplets $\langle\tau_e, \tau_p, \tau_{p_o}\rangle$ and compute their support and confidence (Line 26). The value of support and confidence for each distinct triplet is incremented in each iteration and stored in Ψ_{SUPP} and Ψ_{CONF} maps. Additionally, a map Ψ_{PTT} (Property to Types) is populated with properties and their object types for all the properties having max count equal to one (Line 27).

Finally, in Shapes extraction phase, the algorithm iterates over the values of the Ψ_{CTP} map and defines the *shape name* of s, the *shape's target definition* τ_s, and the set of *shape constraints* ϕ_s for each candidate class (Lines 28-30). The map value MAP$\langle$IRI$_{\text{PROPERTY}}$, SET$\rangle$ for each candidate class is used to extract properties and object types to define property constraints P for each *Node Shape* (Lines 31-43). The property constraints specification for P includes sh:path, and depending on sh:nodeKind either sh:class, or sh:datatype. The sh:nodeKind constraint can possibly have three types of values: sh:Literal, sh:IRI, and sh:BlankNode. In case of Literal types, the literal object type is used, e.g., xsd:string, xsd:integer, or xsd:date. However, in the case of non-literal object types, the constraint sh:class is used to declare the type of object associated with the candidate property. It is possible to have more than one value for the sh:class and sh:datatype constraints of a candidate property shape, in such cases, we use sh:or constraint to encapsulate multiple values.

We provide detailed algorithms for each phase of shape extraction (including definitions of methods invoked in algorithm 1) in appendix A.2 of extended version of the paper.

QSE-Exact (Query-based). To support shapes extraction from a triplestore, we propose a query-based variant of QSE using a set of SPARQL queries [37] to extract all the constraints from a triplestore required to construct the final shape schema $\mathcal{S}$ and compute both the necessary support and confidence. In practice, we pose queries to extract all the distinct classes C, then, for each class $c \in C$, its properties $p \in \mathcal{P}$ along with object types are extracted as triplets, and support is computed for each triplet by a count query.

Shapes Pruning. QSE uses the values of support and confidence of shape constraints to produce shapes satisfying the minimum threshold provided by the user. To output complete shapes, QSE assures that all Property Shapes (PS) are associated with at least one Property Shape Constraint (PSc) and that all Node Shapes (NS) are associated with at least one property shape. To do so, the threshold ε for confidence and ω support (provided by the user) is provided as input to Algorithm 1, and while iterating over the PSc in fourth phase (Lines 28-44), QSE omit creating PSc not satisfying the threshold of support and confidence (Line 35); as a result, PS having no constraints are also pruned (as a cascading effect), and finally, NS having zero PS are also pruned (line 42).

Cardinality Constraints. QSE supports assigning cardinality constraints (sh:minCount and sh:maxCount) to C_{p} for each shape constraint ϕ_s:$\langle\tau_{\text{p}}, \text{T}_{\text{p}}, \text{C}_{\text{p}}\rangle$. Following the open-world assumption, all shape constraints are initially assigned a minimum cardinality of 0, making them optional. However, it is of more value if an accurate cardinality constraint is assigned to a shape. For instance, it makes sense to assign minimum cardinality 1 to name property shape in Listing 1b. A trivial way to include a minimum cardinality 1 in a

Algorithm 2 Neighbors-based Dynamic Reservoir Sampling

Input: Graph $\mathcal{G}$ from File, maximum entity threshold $\tau_{\max}$, Sampling$_{\%}$
Output: Ψ_{ETD} , Ψ_{CEC}
1: init maps Ψ_{ETD} , Ψ_{SEPC}, Ψ_{RCPC} , Ψ_{CEC}, Ψ_{PC}
2: $\tau_{\min}$ = 1 (minimum entity threshold) ; lineCounter = 0
3: **for** t $\in \mathcal{G}$ **do** ▷ parse s,p,o of the triple t
4: **if** t.p = Type Predicate **then**
5: entity e : t.s ; entityType e_t = t.o ▷ s: subject, o: object
6: Ψ_{SEPC}.putIfAbsent(e_t, ...) ▷ if $e_t \notin \Psi_{\text{SEPC}}$
7: Ψ_{RCPC}.putIfAbsent(e_t, $\tau_{\min}$) ▷ if $e_t \notin \Psi_{\text{RCPC}}$
8: **if** $|\Psi_{\text{SEPC}}.\text{get}(e_t)| < \Psi_{\text{RCPC}}.\text{get}(e_t)$ **then** ▷ Add entity e in reservoir
9: **if** $\Psi_{\text{ETD}}.\text{get}(e).\text{T}$ is $\emptyset$ **then** Ψ_{ETD}.insert(e, ...)) ▷ T : entity types
10: Ψ_{ETD}.insert(e, Ψ_{ETD}.get(e).T.add(t.o))
11: Ψ_{SEPC}.get(e_t).insert(e)
12: **else** ▷ Replace random entity in reservoir with current entity e
13: r = generateRandomNumber(0, lineCounter)
14: **if** $r < |\Psi_{\text{SEPC}}.\text{get}(e_t)|$ **then**
15: $\overleftarrow{n}, \widehat{n}, \vec{n} = \Psi_{\text{SEPC}}.\text{get}(e_t)$.nodeAtIndex($r-1, r, r+1$))
16: n = getNodeWithMinimumScope($\overleftarrow{n}, \widehat{n}, \vec{n}$)
17: replace node at index n with current e & e_t in Ψ_{ETD}
18: $\Psi_{\text{SEPC}}.\text{get}(e_t)$.add ($e$)
19: increment entity count for current e_t in Ψ_{CEC}
20: ▷ Resize reservoir
21: ratio = $(\Psi_{\text{SEPC}}.get(e_t).size()/\Psi_{\text{CEC}}.get(e_t)) \times 100$
22: capacity = Sampling$_{\%}$ $\times$ $\Psi_{\text{SEPC}}.\text{get}(e_t)$.size()
23: **if** capacity $< \tau_{max} \wedge$ ratio $\leq$ Sampling$_{\%}$ **then** Ψ_{RCPC}.insert(e_t , capacity)
24: **else**$\rightarrow$ increment property count for current $t.p$ in Ψ_{PC}
25: lineCounter + +

property shape is to assign it to property shapes having confidence 100%, i.e., for those cases in which all entities have that property. In case of incomplete KGs, QSE allows users to provide a different confidence threshold value for adding min cardinality constraints in property shapes. To achieve this, we extend fourth phase and add a min cardinality constraint in property shapes on line 42 based on the min-confidence provided by the user. For instance, if a user provides confidence 90% to add min cardinality constraint, QSE will include a min cardinality constraint equal to 1 to all the property shapes that appear in at least for 90% of the entities. QSE also keeps track of properties having maximum cardinality equal to 1 in second phase and assigns sh:maxCount = 1 to those property shapes in the fourth phase while constructing shapes.

5 QSE-APPROXIMATE

Here we provide an approximate solution to Problem 1, to enable shape extraction from very large KGs. There does not exist any approximate scalable solution for extraction of validating shapes from very large knowledge graphs in an appropriate limited amount of time on a commodity machine. SheXer [11], however, supports shapes extraction from a small dump of WikiData from 2015 (42 GB), but it requires 38.3 hours to extract ShEx shapes using SheXer. The *approximate QSE solution* tackles this limitation by proposing a **N**eighbors-**b**ased **D**ynamic **R**eservoir **S**ampling (NbDRS) approach to sample entities of very large knowledge graph prior to shapes extraction. Algorithm 2 shows the pseudocode for the first step where graph $\mathcal{G}$ is sampled using NbDRS. It requires graph $\mathcal{G}$ file, sampling percentage (Sampling$_{\%}$), and threshold for maximum number of entities to be sampled per reservoir, i.e., τ_{max} maximum size of the reservoir. It starts by initializing hash maps and variables (Lines 1-2) required for storing extracted information while parsing each $\langle s, p, o \rangle$ triple t from a file in the loop (Line 3-25). Triples are filtered based on their type predicate (such as rdf:type and wdt:P31) to extract entities' information (Line 4-24), while non-type triples are filtered on Line 24 to keep count of distinct properties using Ψ_{PC} (Property Count) map.

In NbDRS approach, a reservoir is maintained for each distinct entity type e_t (e.g., reservoir for :Student and :FullProfessor in Figure 1) using a map called Ψ_{SEPC} (Sampled Entities per class). The map Ψ_{RCPC} is designated to store reservoir's capacity for each e_t and allows to increase its capacity dynamically. The e_t is inserted in Ψ_{SEPC} and Ψ_{RCPC} maps for the first time (line 6-7), if the reservoir for the current e_t has capacity, it is filled with its entity information using Ψ_{ETD} map (Lines 8-11); otherwise a random number r is generated between zero and the current line number of the file (Line 13). Here we use the term entity and node interchangeably. The node at index n is called focus node $\widehat{n}$, its left neighbor node $\overleftarrow{n}$ and right neighbor node $\vec{n}$ are identified to choose n as the node with minimum scope. The scope of a node represents the number of types of a specific entity e, e.g., an entity can have multiple types like Person, Student or a Researcher. The node with a minimum scope is replaced in the reservoirs with the current e (Line 17) and Ψ_{SEPC} is updated accordingly (Line 18). The real entity count of each type is also computed and stored in Ψ_{CEC} (Class to Entity Count) map. This reservoir sampling approach is called dynamic as it computes the current sampling ratio (with respect to sampled entities and the total entities of the current type - Line 21) and new size of the reservoir using sampling percentage and count of sampled entities (Line 22). If the newly computed size is less than τ_{max} and the current sampling ratio is less than the specified sampling percentage, then the reservoir capacity of the current type is increased by the newly computed reservoir's capacity (Line 23). Once the sampling is performed, the sampled entities of $\mathcal{G}$ are used in the same way as in second phase of Algorithm 1 (From line 9) to extract their property information required for shape extraction in the subsequent algorithms. We provide a detailed version of Algorithm 2 in appendix A.3 of extended version of the paper.

While performing shapes pruning using sampled entities, the only difference as compared to exact QSE solution is that approximate QSE solution requires to compute relative support ω' and confidence ε' of property shapes. The relative ω' is computed using the following formula shown in equation 3 where ω is the provided minimum support threshold, P represents all triples in $\mathcal{G}$ having property P, P_r* represents triples having property P across all entities in in all reservoirs, T represents all entities of type T in $\mathcal{G}$, and T_r represents all entities of type T in the reservoir. Similarly, relative confidence ε' of a property shape is computed by replacing the denominator in equation 2 with $|T_r|$.

$$\omega' = \omega \times min(\frac{|P_r*|}{|P|}, \frac{|T_r|}{|T|}) \tag{3}$$

In order to compute the quality of our NbDRS approach, we compute precision and recall of Node Shapes (NS) and Property Shapes (PS) without sampling (extracted using exact QSE); and with sampling, i.e., NS′ and PS′ (extracted using approximate QSE). Precision and recall of node shapes is computed using equation 4 while for property shapes is computed using equation 5. These formulas are inspired from the context of information retrieval.

$$\text{Precision}_{NS} = \frac{|NS \cap NS'|}{|NS'|}, \text{Recall}_{NS} = \frac{|NS \cap NS'|}{|NS|} \tag{4}$$

$$\text{Precision}_{PS} = \frac{|PS \cap PS'|}{|PS'|}, \text{Recall}_{PS} = \frac{|PS \cap PS'|}{|PS|} \quad (5)$$

6 EVALUATION

In this section, we empirically analyze QSE approach to demonstrate the problem of *spuriousness* – the effect of pruning and how QSE solves the spuriousness, and the *scalability* of exact and approximate QSE with further evaluation of shapes extracted using approximate solution.

Datasets. We selected one synthetic and three real-world datasets: LUBM [15], DBpedia [3], YAGO-4 [50], and WikiData [53]. In particular, we used LUBM-500, DBpedia downloaded on 01.10.2020, for YAGO-4, we used the subset containing instances having English Wikipedia articles downloaded on 01.12.2020, and for WikiData, we used two variants, i.e., Wdt15 – a dump from 2015 [57] used by one of the related work SheXer [11], and Wdt21 – the truthy dump from September 2021 filtered using a python script to remove labels and descriptions along with strings having language other than English. The python script and other related details to download these datasets are available on our GitHub [39] repository An overview of the characteristics of each dataset is shown in Table 1. The datasets differ w.r.t. their sizes (on disk in gigabytes and number of triples), the number of distinct classes, properties, subjects, and objects.

Implementation and Setup. We have implemented QSE algorithms in JAVA-11. QSE supports both shapes extraction from RDF files (NT or N3 format) and a triplestore. All experiments are performed on a single machine with Ubuntu 18.04, having 16 cores and 256 GB RAM. We have used GraphDB [14] 9.9.0 to experiment with the QSE-Exact (query-based) with a maximum memory usage limit set as 16 GB. Experiments to measure running time are executed three times, and the average runtime is computed.

Source Code and Resources. The source code of our approach is available as open-source [39] along with experimental settings and datasets. We have published the extracted SHACL shapes of all our datasets on Zenodo [46].

Metrics. We measure the *running time* (in minutes) to extract the shapes for a given dataset (file or triplestore), maximum *memory usage* during the extraction process, and *Shape Statistics* of the output shapes. Additionally, we also measure the time required to load each dataset in a triplestore to experiment with query-based QSE-Exact approach.

6.1 Performance Analysis

Exact QSE. We use QSE-Exact approach to extract SHACL shapes from LUBM, DBpedia, YAGO-4, and WikiData datasets. The statistics of the shapes extracted from these datasets using QSE-Exact (file-based) is shown in Table 2. It shows the count of Node Shapes (NS), Property Shapes (PS), and Property Shape Constraints (PSc) - literal and non-literal for all datasets. We consider ShapeDesigner [4], SheXer [11], and SHACLGEN [20] as state-of-the-art approaches to compare against QSE by measuring shapes extraction time and memory consumption. Figure 2 shows the running time and Table 3 shows the memory consumption to extract shapes for DBpedia, LUBM, YAGO-4, and Wdt15 datasets using QSE-Approximate (file-based), QSE-Exact (file-based and query-based) approach, and SheXer. The memory limits for QSE are defined using Java Xmx parameter.

The QSE-Approximate (file-based) approach took 1, 2, 13, 13 minutes to extract SHACL shapes from DBpedia, LUBM, Wdt15, and YAGO-4 respectively; and 10 GB RAM for all datasets except Wdt15 which required 16 GB of RAM due to high number of distinct subjects. The maximum reservoir size was set to 1000 with 100% target sampling percentage for sampling in QSE-approximate. The QSE-Exact (file-based) approach took 3, 8, 16, 23 minutes to extract SHACL shapes from DBpedia, LUBM, Wdt15, and YAGO-4 respectively; and 16 GB RAM for all datasets except Wdt15 which required 50 GB of RAM due to high number of distinct subjects.

The QSE-Exact (query-based) approach took 34, 47, 72, 148 minutes to extract SHACL shapes from DBpedia, LUBM, Wdt15, and YAGO-4 respectively; and 16 GB RAM for all datasets. It took 17 minutes to load DBpedia, 23 minutes to load LUBM, 78 minutes to load YAGO-4, and 71 minutes to load and Wdt15 into GraphDB triplestore in our setup. Both ShapeDesigner and SHACLGEN load the whole graph into a triplestore similar to our QSE-Exact (query-based) approach to extract shapes. We report the shape extraction time for existing query-based approaches [4, 20] as of our QSE-Exact (query-based) approach, and consider that they will take more or less the same amount of time to for shape extraction. Although *ShapeDesigner* is a UI-based tool that can not handle large datasets with a few million triples (also requires human intervention to extract shapes) and *SHACLGEN* is not suited to extract shapes of datasets having hundreds of classes as it required multiple hours to generate shapes for datasets having many classes such as YAGO-4 (8,897 classes). Both ShapeDesigner and SHACLGEN are in-memory-based approaches as they try to load the whole graph in the main memory to extract shapes, which is also one of the main reasons that they do not support shapes extraction from very large graphs, especially on a commodity machine.

SheXer took 26, 58, 195, 117 minutes to extract SHACL shapes from DBpedia, LUBM, Wdt15, and YAGO-4 respectively; and 18, 33, 24, 59.8 GB RAM respectively. The acceptance threshold was set to 100 for SheXer to extract all shapes from the datasets. Wdt15 is the dataset used in SheXer's experiments, we have used it without applying any kind of data curation as oppose to the way classes are filtered in SheXer.

The QSE-Exact (file-based) is the most efficient approach with zero overhead or requirement to load the graph in a triplestore as compared to other exact approaches like QSE-Exact (query-based) and SheXer. As Wdt21 is a very large dataset, the QSE-Exact approach took 2.56 hours and 235 GB RAM to extract its SHACL shapes while SheXer could not output shapes within 24 hours on our machine having 256 GB RAM, therefore, we interrupted the experiment. Extracting shapes from large datasets like Wdt15 and

Table 1: Size and characteristics of the datasets

	DBpedia	LUBM	YAGO-4	Wdt15	Wdt21
# of triples	52 M	91 M	210 M	290 M	1.926 B
# of distinct objects	19 M	12 M	126 M	64 M	617 M
# of distinct subjects	15 M	10 M	5 M	40 M	196 M
# of distinct literals	28 M	5.5 M	111 M	40 M	904 M
# of distinct instances	5 M	1 M	17 M	3 M	91 M
# of distinct classes	427	22	8,902	13,227	82,693
# of distinct properties	1,323	20	153	4,906	9,017
Size in GBs	6.6	15.66	28.59	42	234

Table 2: Shapes Statistics using QSE-Exact.

	NS	PS	Non-Literal PSc	Literal PSc
	COUNT	COUNT/AVG	COUNT/AVG	COUNT/AVG
LUBM	23	164 / 7.13	323 / 3.02	57 /1
DBpedia	426	11,916 / 27.97	38,454 / 6.96	5,335 /1
YAGO-4	8,897	76,765 / 8.62	315,413 / 14.54	50,708 / 1
Wdt15	13,227	202,085 / 15.27	114,890 / 3	106,599 / 1
Wdt21	82,651	2,051,538 / 24.82	3,765,953 / 5.651	1,113,856 / 1

Wdt21 is not feasible on a commodity hardware using exact approaches. Our QSE-Approximate approach provides a solution to deal with this limitation and allows shape extraction from very large datasets. We discuss the results of using QSE-Approximate on Wdt21 in the next section.

Approximate QSE. Here we measure the performance of the approximate QSE approach to extract shapes from Wdt21. As explained in Section 5, NbDRS algorithm 2 allows users to specify the sampling percentage and upper limit of the reservoir size (maximum number of entities to be sampled per class). We show the results of applying NbDRS algorithm on Wdt21 in Table 4. It shows for both, Node Shapes (NS) and Property Shapes (PS), the resulting number of sampled NS and PS along with their precision, recall, relative error, and the reservoir size. The target sampling percentage was set to 100% and the maximum memory limit was set to Xmx 32 GB in JAVA VM. The QSE-Approximate approach is able to sample all the NS with zero relative error, however, the resulting sampled PS varies depends on the maximum size of the reservoir.

Table 3: Memory consumption analysis

	DBpedia	LUBM	YAGO-4	Wdt15
SheXer	18 GB	33 GB	24 GB	59 GB
QSE-Exact (File-based)	16 GB	16 GB	16 GB	50 GB
QSE-Exact (Query-based)	16 GB	16 GB	16 GB	16 GB
QSE-Approximate (File-based)	10 GB	10 GB	10 GB	16 GB

Table 4: Sampling results on Wdt21 showing number of real and sampled NS, PS, their Precision (P), Recall (R), and Relative Error with 100% sampling percentage.

Reservoir Size	Node Shapes (NS)				Property Shapes (PS)			
	Real	Sampled	P / R	Error	Real	Sampled	P / R	Error
250	82,651	82,651	1 / 1	0	2,051,538	1,824,342	1 / 0.89	227,196
500	82,651	82,651	1 / 1	0	2,051,538	1,896,826	1 / 0.92	154,712
750	82,651	82,651	1 / 1	0	2,051,538	1,917,830	1 / 0.93	133,708
1,000	82,651	82,651	1 / 1	0	2,051,538	1,937,623	1 / 0.94	113,915
1,250	82,651	82,651	1 / 1	0	2,051,538	1,947,223	1 / 0.95	104,315
1,500	82,651	82,651	1 / 1	0	2,051,538	1,957,753	1 / 0.95	93,785

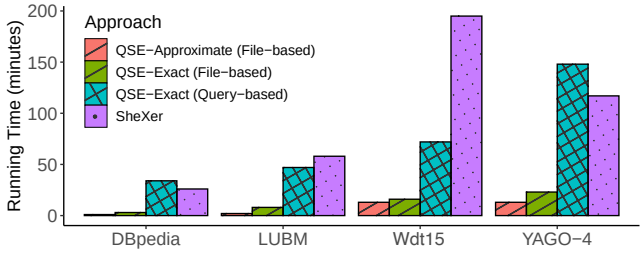


Figure 2: Running time analysis

By maximum here we mean that the reservoir's size starts from 1 and can go up to the specified maximum limit. The relative error for PS decreases by increasing the upper limit of the reservoir's size and Recall increases respectively. We omit stating the results of QSE-Approximate using different target sampling percentage due to space limitation, however, more results can be found on our website.

6.2 Computational Complexity Analysis

The complexity of QSE-Exact approach (Algorithm 2) is computed as $O(t_E + t_{\mapsto E} + E \times \Phi_s + S \times \Phi_s)$. Here, t_E represents entity type triples (extracted in the first phase), $t_{\mapsto E}$ represents those triples where subject maps to entities (extracted in the second phase), E represents entities (i.e., the set of distinct IRIs that appear as subject for some triple), S represents number of Node Shapes, and lastly, Φ_s represents a set of all property shape constraints, i.e., $\Phi_s = \{\phi_1, \phi_2, ..., \phi_n\}$ where $\phi_s{:}\langle \tau_p, T_p, C_p \rangle$, see definition 3.2. For QSE-Approximate approach, the complexity of Algorithm 2 is computed as $O(t)$ where t represents triples. The Algorithm 2 replaces the first phase over the file of Algorithm 1 to collect a limited number of entities using neighbor-based dynamic reservoir sampling technique. This indicates that overall complexity of shape extraction reduces when using QSE-Approximate due to reduction in number of entities $|E|$.

The computational complexity of SheXer [11] is computed as $O(t + t_E + t_{\mapsto E} + \{({\Phi_s}^2/S) \times E\})$. For example, to compute the computational complexity for shape extraction from Wdt21, we can use the statistics from table 1 and 2. To keep it simple, we ignore the number of triples t, and use other values like $E = 91M$, $S = 82,651$, $\Phi_s = 2M$ having literal and non literal constraints $4.8M$. The results show that the complexity of QSE-Exact is lower than SheXer by one order of magnitude. We provide exact numbers and calculations on our website. Should we? Ask Matteo!

6.3 Qualitative Analysis

To deal with the issue of spuriousness, we perform qualitative analysis on the extracted shapes by performing pruning which is the process where QSE *support-based shapes extraction* comes into action after extraction of all shapes of a given dataset and prunes out the shapes a minimum support and confidence specified by the user. This applies to both, QSE-Exact and QSE-Approximate approaches. Pruning of Property Shapes constraints (PSc) results in the pruning of Property Shapes (PS), and pruning of PS causes pruning of Node Shapes (NS) as a cascading effect; therefore, we also analyze the pruning of PS and NS. To study the impact of various confidence and support thresholds on the number of PSc, PS, and NS, we analyze the effect of pruning by specifying values for confidence greater than $(25, 50, 75, 90)\%$ and support $(\geq 1, >100)$. For instance, given a confidence value $>25\%$ and support >100, for every PS, QSE prunes all the PSc where support is not greater than 100 and confidence is not greater than 25%.

DBpedia. Figure 3 shows the result of pruning PSc (3a), PS and NS (3e) for our specified range of confidence and support on DBpedia. In general, the results show that higher the confidence and the threshold for the support, the higher is the percentage of PSc and PS to be pruned. For instance, DBpedia contains 922 PSc

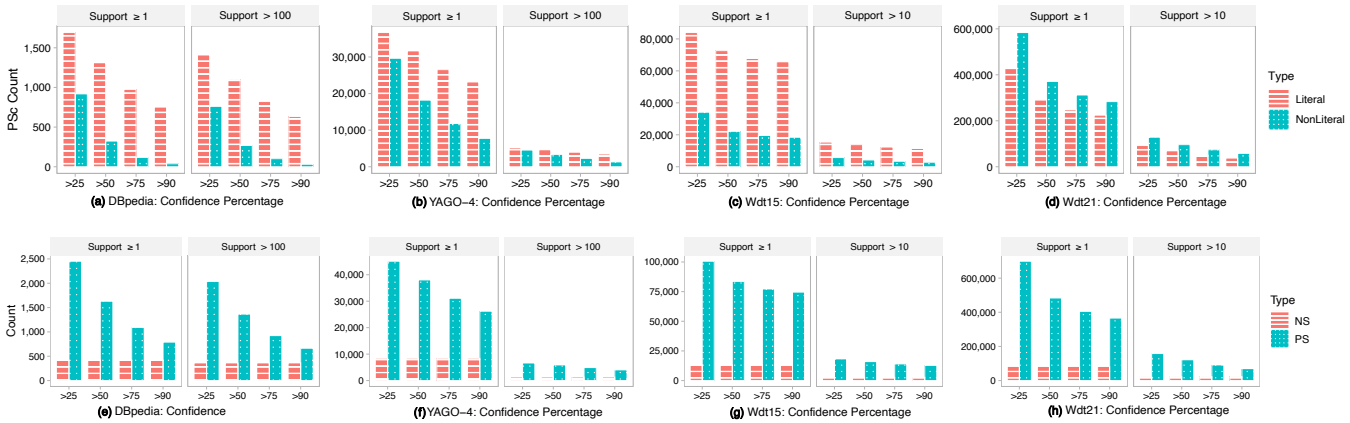


Figure 3: Qualitative analysis of QSE-Exact (file-based) approach on DBpedia, YAGO-4, and WikiData (2015 and 2021 dumps)

(specifically non-literal sh:class constraints) for confidence > 25% and support ≥ 1. When we keep the confidence value at > 25% and increase the support, QSE prunes out 161 constraints, by increasing support to > 100. Analogously, we see a similar trend in the number of pruned constraints by increasing confidence to > 50%, 75%, and 90% against each value of support. In comparison to the default shape statistics (Table 2) for PSc and PS, pruning resulted in a drastic decrease in the number of PSc and PS. The majority of the non-literal PSc are pruned out compared to literal PSc; this shows that most of the spurious constraints are non-literal. Pruning of NS is lower compared to PS and PSc for all combinations of support and confidence. We show the statistics about the number of entities of each class in DBpedia (for our downloaded version). The resulting shape schema for DBpedia contains 2,471 PS after pruning using threshold of confidence > 25% and support ≥ 1 (Figure 3e), which shows a reduction of 1 order of magnitude as a result of pruning in comparison to the default shape statistics for PS (Table 2). Similarly, increasing the confidence threshold to > 90% for support ≥ 1 results in the pruning of up to 2 orders of magnitude.

YAGO-4. Figure 3 shows the result of PSc (3b), PS and NS (3f) pruning on YAGO-4. Our results show that applying pruning with confidence > 25% and support ≥ 1 on YAGO-4 results in the pruning of the PSc (non-literal) up to 10x compared to default shape statistics (Table 2). Increasing the support threshold to > 100 with confidence > 25% shows an interesting sudden decrease in the number of PSc of two orders of magnitude. This shows an interesting fact about entities in YAGO-4, i.e., a large portion of its taxonomy contains fewer than 100 entities. More interestingly, our results show a consistent decrease in the number of constraints when increasing the confidence threshold for all support values. Analogously to these results, Figure 3f shows the same trend for pruning of PS.

WikiData. Figure 3 shows the result of pruning on Wdt15 and Wdt21 using QSE-Exact. As these are large datasets, we specify the values of support as (≥ 1, >10) and confidence greater than (25, 50, 75, 90)% to study the effect of pruning in details. We noticed the same trend as other datasets that pruning using support helps to prune out a very large number of shape constraints which are representative of very small portion of the dataset. For example, given a confidence value >25% and support >10, for every PS, QSE prunes all the PSc where minimum support is not 10 and confidence > 25% does not hold. This shows approximately 5x reduction in the number of literal constraints and 3x reduction in the number of literal constraints (3c,d). Similarly, we notice 3.5x and 2x reduction in number of PS and NS for Wdt15 (3g); and 2.5 and 2x reduction in number of PS and NS for Wdt21 (3h) given support >10 and confidence >25% respectively. Upon increasing the threshold for confidence and support, it follows the same smooth decreasing trend.

Table 5 shows the results of pruning on shapes extracted from Wdt21 using QSE-Approximate having statistics shown in Table 4 (Line 2). We started with pruning using confidence 25% and support varying from 1 to 100. The values in *Real* column shows the number of NS or PS extracted using QSE-Exact (without sampling), while the values in *Sampled* column shows the number of NS or PS extracted using QSE-Approximate (with sampling) pruned with the specified threshold for confidence and support. Additionally, in QSE-Approximate, the relative support is computed using the equation 3 for each property to perform pruning relative to the size of sampled entities. We noticed that with support ≥ 1 and confidence > 25%, QSE-approximate is able to get almost the same number of PS as in case of QSE-Exact for Wdt21 (Figure 3g) having recall as 0.89 and precision equal to 1. Additionally, upon increasing the support to 10, 25, 50, and 100, we notice a constant recall of 88% and slight reduction in precision, i.e., 98% and 96% with decreasing relative error. Similarly, we notice the same trend with confidence values greater than 50, 75, and 90%.

NbDRS vs. Standard Reservoir Sampling. We studied the difference between *neighbor-based dynamic reservoir sampling* and *standard reservoir sampling* by running Algorithm 2 with and without neighbor-based-dynamic reservoir sampling for various values of sampling percentage (i.e., 25, 50, 75, 100) and maximum size of reservoir (i.e., 500, 1000, 1500). The results showed that NbDRS sampling approach for shape extraction is able to extract in between 4K to 25K more property shapes for different node shapes as compare to standard reservoir sampling where reservoir size stays constant and the replacement of elements in the reservoirs does not take into account the neighbor elements.

Table 5: Sampling results on Wdt21 using QSE-Approximate showing number of real and sampled NS, PS their Precision (P), Recall (R), and Relative Error for 500 reservoir size with 100% sampling percentage.

Conf	Supp	Node Shapes (NS)				Property Shapes (PS)				
		Real	Sampled	P / R	Error	Real	Sampled	P	R	Error
> 25 %	≥ 1	82,651	82,651	1	0	698,825	620,622	1	0.89	78,203
	10	23,640	23,640	1	0	158,283	141,040	0.99	0.88	17,243
	25	14,683	14,683	1	0	95,333	85,450	0.98	0.88	9,883
	50	9,920	9,920	1	0	62,038	55,963	0.98	0.88	6,075
	100	6,596	6,596	1	0	39,877	36,362	0.96	0.88	3,515
> 50 %	≥ 1	82,651	82,651	1	0	483,813	431,123	1	0.89	52,690
	10	23,640	23,640	1	0	121,866	109,301	0.99	0.89	12,565
	25	14,683	14,683	1	0	74,694	67,454	0.99	0.89	7,240
	50	9,920	9,920	1	0	48,997	44,496	0.98	0.89	4,501
	100	6,596	6,596	1	0	31,509	28,949	0.97	0.89	2,560
> 75 %	≥ 1	82,651	82,651	1	0	405,344	362,717	1	0.89	42,627
	10	23,640	23,640	1	0	91,947	83,329	0.99	0.9	8,618
	25	14,683	14,683	1	0	56,744	51,769	0.99	0.9	4,975
	50	9,920	9,920	1	0	37,328	34,251	0.98	0.9	3,077
	100	6,596	6,596	1	0	23,944	22,193	0.97	0.9	1,751
> 90 %	≥ 1	82,651	82,651	1	0	366,045	327,721	1	0.89	38,324
	10	23,640	23,640	1	0	69,951	63,802	0.99	0.9	6,149
	25	14,683	14,683	1	0	43,196	39,706	0.99	0.91	3,490
	50	9,920	9,920	1	0	28,371	26,265	0.98	0.91	2,106
	100	6,596	6,596	1	0	18,028	16,891	0.97	0.91	1,137

We have presented the results of QSE approach on LUBM dataset and results of *assigning cardinality constraints using support and confidence* in appendix B.1 and B.2 of extended version of the paper due to space limitations.

7 RELATED WORK

Integrity constraint semantics were initially defined in OWL [29, 30, 51]. OWL proved to be not well-suited for expressing integrity constraints as it was designed to model incomplete data assuming the open-world assumption. Later, the SPARQL Inferencing Notation (SPIN) [23] was proposed to enforce constraints using SPARQL queries. SHACL [24] is known as the next generation of SPIN and became a W3C recommended language for defining integrity constraints over RDF in 2017. Similar to SHACL, ShEX [36] is a constraint language that is built on regular bag expressions inspired by schema languages for XML. Nowadays, validating schemas like SHACL and ShEx are also being used for purposes other than validation, such as building user interfaces, optimizing query processing [40], and automatically generating forms to populate RDF datasets [18].

SHACL is a W3C standard which models integrity constraints against specific RDF target resources (entities and attributes) in the form of *node* and *property* shapes, whereas integrity requirements on properties associating multiple targets are expressed using links between shapes. ShEx, instead, defines constraints in terms of shape expressions, and while it is not a standard, it is used within the WikiData project [52]. Even though SHACL and ShEx are not completely equivalent [13], their core mechanism revolves around the concept of enforcing for each node in the KG to satisfy specific constraints on the combination of its types and the connected predicates [11].

The former provides interactive graphical editors that allows a domain expert to *manually specify* the constraints that each shape needs to enforce for some portion of the data, and the later [11, 20] derive shape constraints from RDF instance data within a KG, which we call the process of *shape extraction*. This process identifies in the KG co-occurrence of types and predicates and computes their cardinalities. The implementation of the shape extraction process varies across different approaches, e.g., SHACLGEN [20] and ShapeDesigner [4] are query-based approaches, i.e., they load the input graph in a triplestore in order to query its contents, SheXer [11] parses the input graph file directly after loading it completely into memory, and Astrea [7] only supports extracting shapes from existing ontologies instead of instance data. Because of their implementation, existing semi-automatic approaches for shape extraction are cumbersome since they require manual intervention and automatic approaches require days to extract shapes from very large KGs. Furthermore, in our recent survey [41] on extraction and adoption of validating shapes we show that *there is a growing need to develop techniques for efficient extraction of validating shapes from very large existing KGs.*

Applications. Validating schemas have been adopted for purposes other than validation as well. Specifically, for optimizing query processing in KGs [40], building and automatically generating forms to populate RDF datasets [18, 58], type and compliance checking [26, 42], intelligent geoprocessing [19], automatic detection of metadata errors in clinical studies [22], and for customization of Schema.org [47].

Shape Extraction. There has been recent development to derive validating shapes from existing KG instance data; various applications are created to assist the process of shapes extraction, which can potentially be divided into two phases, i.e., preprocessing and shapes construction. The *preprocessing* phase involves collecting information from the input dataset about classes and their properties, which are then used in the *shapes construction* phase. The preprocessing phase can be conducted in two alternative ways: query-based and non-query-based. The query-based solutions involve loading the KG into a triplestore able to answer SPARQL queries. The triplestore is then used to access the information required for the shapes construction phase. SHACLGEN [20], RDFShape [56], and ShapeDesigner [4] are query-based solutions to generate SHACL or ShEx shapes from existing RDF data. Non-query-based solutions, instead, parse RDF data stored as files on disk. SheXer [11] is the only tool to generate ShEx shapes that supports both a triplestore and RDF files as input. Spahiu et al. [48] proposed a solution based on a data profiling tool called ABSTAT [49] to generate semantic profiles and transform them into SHACL to improve the quality of the KGs. Mihindukulasooriya et al. [28] proposed a KG data profiling-based RDF shape induction approach by using predictive modeling to generate shapes. SHACLearner [32] is a method to learn SHACL constraints based on Inverse Open Path rules (IOP) rules. Astrea [7] is an ontology-based approach to extract SHACL from ontologies. Finally, RML2SHACL [10] also generates SHACL shapes but it requires RML mappings. We have classified existing approaches in Table 6 based on their features, i.e., support for shapes extraction from data or ontologies, support for automatic extraction of shapes, support for shapes extraction from a SPARQL triplestore, and whether they extract SHACL, ShEx, or both types of validating shapes. Among all the state-of-the-art approaches, some tools support extraction from ontologies only (such as Astrea [7]), others support extraction from instance data, and very few support automatic generation of shapes from both ontology and instance

Table 6: State-of-the-art to extract validating shapes [41]

Approach	*Extracted from*		*Auto-matic*	*Triple-store*	*Type*
	data	*ontology*			
Shape Induction [28]	✔	✗	✔	✔	SHACL,ShEx
SheXer [11]	✔	✗	✔	✔	SHACL,ShEx
Spahiu et al. [48]	✔	✗	✔	✔	SHACL
ShapeDesigner. [4]	✔	✗	✔	✔	SHACL,ShEx
SHACLGEN [20]	✔	✔	✔	✔	SHACL
TopBraid [38]	✔	✔	✔	✔	SHACL
Pandit et al. [34]	✗	✔	✗	✔	SHACL
Astrea [7]	✗	✔	✔	✗	SHACL
SHACLearner [32]	✔	✗	✔	✗	SHACL

data. Methods that generate shapes from ontologies assume the provided ontology to be complete (i.e., providing complete coverage of the instances in the KG and their properties) and to be of small size (they do not expect more than a few hundred classes). In contrast to our work, as motivated in the Introduction 1, the existing automatic and semi-automatic shapes extraction has two common limitations, i.e., scalability and spuriousness. Our approach aims to overcome this limitations.

Rule Discovery and Pattern Mining. There exists various approaches for rule discovery in graphs [27]. These systems [12, 33] derive rules from large KGs using structural information by exploring the frequently occurring graph patterns. RuleHub [1] provides a public corpus of rules for KGs, and also allows to retrieve new rules and refine the corpus. Some approaches also report statistical measures such as confidence for the discovered positive and negative rules [1]. In contrast to our approach of shape extraction, such derived rules are mainly used to derive new facts, better understand the underlying data in the graphs, and derive conclusions. In frequent subgraph mining, the occurrence of subgraphs (the number of times a subgraph appears) is taken as the support of subgraphs in large graphs. The most practical measurement for measuring this support is minimum image-based support (MNI [6]). Our proposal of defining support for shape constraints is inspired from the concept of support in MNI [6] and frequent pattern mining [16].

Sampling and Summarization. There exists various sampling and summarization approaches for KGs applied to different problem domains. Debattista et al. [9] apply reservoir sampling approach to to approximate specific quality metrics (dereferenceability of URIs and links to external data providers) for large and evolving datasets. In contrast to our work, we propose dynamic neighbor-based reservoir sampling to sample the entities of very large graphs for shape extraction. Similarly, ABSTAT [2] is a summarization method for RDF graphs aims at reflecting relationships of class instances and their properties. The output summaries are abstract knowledge patterns built in polynomial time to the size of the input graph. Pham et al. [35] combine statistical and structural non-quotient approaches to create *relational schema* summary of the RDF graphs by computing sets of properties joining on the subject – called characteristic sets. Such computations are performed by loading the RDF data in a relational database and executing queries. The goal of such summaries to give the users better understanding of the structure of the RDF graph and this kind of approaches falls under query-based approaches. In contrast to our work, we focus on efficient shape extraction with quality guarantees mainly aimed to enable validation and quality assessment in KGs (RDF graph files). There exists some other approaches [25] for schema inference in property graphs, however, validation in property graphs is not defined using shapes constraints as a standard, therefore, such approaches can not be applied for KGs. For more details, we refer to the survey on semantic schema discovery [21].

We have found DBpedia SHACL shapes extracted from the DBpedia ontology on Zenodo[1]. In contrast to the shapes we have published on Zenodo, our shapes are extracted using datasets, not ontologies.

8 CONCLUSION

We addressed the two common limitations in existing techniques of shapes extraction in KGs, i.e., scalability and spuriousness. The existing techniques of shape extraction requires days to extract shapes from very large KGs and they produce shapes matching spurious triples in the KG, which are typically due to some imputation error. We addressed these limitations by introducing a *support-based shape extraction* approach called QSE (Quality Shapes Extraction) . We devised an exact and an approximate solution to enable efficient extraction of shapes on commodity machines. QSE approach pruned 93% of invalid and spurious shapes resulting in a reduction of 2 orders of magnitude in the number of shape constraints in DBpedia, and 12x reduction in shape extraction time on WikiData. Overall, the experimental analysis showed that QSE enables shape extraction from very large KGs on commodity machines.

In the future, we will investigate the validation results of validating various large KGs against validating schemas produced by QSE and study the possible implication of reasoning in the process of shape extraction to improve their quality.

REFERENCES

[1] Naser Ahmadi, Thi-Thuy-Duyen Truong, Le-Hong-Mai Dao, Stefano Ortona, and Paolo Papotti. 2020. Rulehub: A public corpus of rules for knowledge graphs. *Journal of Data and Information Quality (JDIQ)* 12, 4 (2020), 1–22.

[2] Renzo Arturo Alva Principe, Andrea Maurino, Matteo Palmonari, Michele Ciavotta, and Blerina Spahiu. 2021. ABSTAT-HD: a scalable tool for profiling very large knowledge graphs. *The VLDB Journal* (2021), 1–26.

[3] Sören Auer, Christian Bizer, Georgi Kobilarov, Jens Lehmann, Richard Cyganiak, and Zachary G. Ives. 2007. DBpedia: A Nucleus for a Web of Open Data. In *The Semantic Web, 6th International Semantic Web Conference (Lecture Notes in Computer Science)*, Vol. 4825. Springer, Busan, Korea, 722–735.

[4] Iovka Boneva, Jérémie Dusart, Daniel Fernández-Álvarez, and José Emilio Labra Gayo. 2019. Shape Designer for ShEx and SHACL constraints. In *Proceedings of the ISWC 2019 Satellite Tracks (CEUR Workshop Proceedings)*, Vol. 2456. CEUR-WS.org, Auckland, New Zealand, 269–272.

[5] Christian Borgelt. 2012. Frequent item set mining. *Wiley interdisciplinary reviews: data mining and knowledge discovery* 2, 6 (2012), 437–456.

[6] Björn Bringmann and Siegfried Nijssen. 2008. What is frequent in a single graph?. In *Proceedings of the 12th Pacific-Asia conference on Advances in knowledge discovery and data mining*. 858–863.

[7] Andrea Cimmino, Alba Fernández-Izquierdo, and Raúl García-Castro. 2020. Astrea: Automatic Generation of SHACL Shapes from Ontologies. In *ESWC (Lecture Notes in Computer Science)*, Vol. 12123. Springer, Heraklion, Crete, Greece, 497–513.

[8] WWW Consortium. 2014. RDF 1.1. https://w3.org/RDF/. Accessed 10th February.

[9] Jeremy Debattista, Santiago Londoño, Christoph Lange, and Sören Auer. 2015. Quality assessment of linked datasets using probabilistic approximation. In *European semantic web conference*. Springer, 221–236.

[10] Thomas Delva, Birte De Smedt, Sitt Min Oo, Dylan Van Assche, Sven Lieber, and Anastasia Dimou. 2021. RML2SHACL: RDF Generation Taking Shape. In *K-CAP*. ACM, USA, 153–160.

[1] https://zenodo.org/record/4021024

[11] Daniel Fernandez-Álvarez, Jose Emilio Labra-Gayo, and Daniel Gayo-Avello. 2022. Automatic extraction of shapes using sheXer. *Knowledge-Based Systems* 238 (2022), 107975.

[12] Luis Galárraga, Christina Teflioudi, Katja Hose, and Fabian M Suchanek. 2015. Fast rule mining in ontological knowledge bases with AMIE+. *The VLDB Journal* 24, 6 (2015), 707–730.

[13] Jose Emilio Labra Gayo, Eric Prud'Hommeaux, Iovka Boneva, and Dimitris Kontokostas. 2017. Validating RDF data. *Synthesis Lectures on Semantic Web: Theory and Technology* 7, 1 (2017), 1–328.

[14] GraphDB. 2022. GraphDB. https://graphdb.ontotext.com. Accessed 20th January.

[15] Y. Guo, Z. Pan, and J. Heflin. 2005. LUBM: A benchmark for OWL knowledge base systems. *Journal of Web Semantics* 3 (2005), 158–182.

[16] Jiawei Han, Hong Cheng, Dong Xin, and Xifeng Yan. 2007. Frequent pattern mining: current status and future directions. *Data mining and knowledge discovery* 15, 1 (2007), 55–86.

[17] Daniel Hernández, Luis Galárraga, and Katja Hose. 2021. Computing How-provenance for SPARQL queries via Query Rewriting. *Proceedings of the VLDB Endowment* 14, 13 (2021), 3389–3401.

[18] Aidan Hogan. 2020. Book. In *The Web of Data*. Springer, Cham.

[19] Zhi-Wei Hou, Cheng-Zhi Qin, A Zhu, Yi-Jie Wang, Peng Liang, Yu-Jing Wang, Yun-Qiang Zhu, et al. 2021. Formalizing Parameter Constraints to Support Intelligent Geoprocessing: A SHACL-Based Method. *ISPRS International Journal of Geo-Information* 10, 9 (2021), 605.

[20] Alexis Keely. 2022. SHACLGEN. https://pypi.org/project/shaclgen/. Accessed 20th January.

[21] Kenza Kellou-Menouer, Nikolaos Kardoulakis, Georgia Troullinou, Zoubida Kedad, Dimitris Plexousakis, and Haridimos Kondylakis. 2021. A survey on semantic schema discovery. *The VLDB Journal* (2021), 1–36.

[22] Daniel Keuchel and Nicolai Spicher. 2021. Automatic Detection of Metadata Errors in a Registry of Clinical Studies Using Shapes Constraint Language (SHACL) Graphs. *Studies in health technology and informatics* 281 (2021), 372–376.

[23] Holger Knublauch, James A Hendler, and Kingsley Idehen. 2011. SPIN-overview and motivation. *W3C Member Submission* 22 (2011), W3C.

[24] Holger Knublauch and Dimitris Kontokostas. 2017. Shapes constraint language (SHACL). *W3C Candidate Recommendation* 11, 8 (2017).

[25] Hanâ Lbath, Angela Bonifati, and Russ Harmer. 2021. Schema inference for property graphs. In *EDBT 2021-24th International Conference on Extending Database Technology*. 499–504.

[26] Martin Leinberger, Philipp Seifer, Claudia Schon, Ralf Lämmel, and Steffen Staab. 2019. Type Checking Program Code Using SHACL. In *ISWC (1) (Lecture Notes in Computer Science)*, Vol. 11778. Springer, Auckland, New Zealand, 399–417.

[27] Michael Loster, Davide Mottin, Paolo Papotti, Jan Ehmüller, Benjamin Feldmann, and Felix Naumann. 2021. Few-shot knowledge validation using rules. In *Proceedings of the Web Conference 2021*. 3314–3324.

[28] Nandana Mihindukulasooriya, Mohammad Rifat Ahmmad Rashid, Giuseppe Rizzo, Raúl García-Castro, Óscar Corcho, and Marco Torchiano. 2018. RDF shape induction using knowledge base profiling. In *Proceedings of the 33rd Annual ACM Symposium on Applied Computing, SAC*. ACM, Pau, France, 1952–1959.

[29] Boris Motik, Ian Horrocks, and Ulrike Sattler. 2007. Adding Integrity Constraints to OWL. In *Proceedings of the OWLED 2007 Workshop on OWL: Experiences and Directions (CEUR Workshop Proceedings)*, Vol. 258. CEUR-WS.org, Innsbruck, Austria.

[30] Boris Motik, Ian Horrocks, and Ulrike Sattler. 2009. Bridging the gap between OWL and relational databases. *Journal of Web Semantics* 7, 2 (2009), 74–89.

[31] Natasha Noy, Yuqing Gao, Anshu Jain, Anant Narayanan, Alan Patterson, and Jamie Taylor. 2019. Industry-scale knowledge graphs: lessons and challenges. *Commun. ACM* 62, 8 (2019), 36–43.

[32] Pouya Ghiasnezhad Omran, Kerry Taylor, Sergio José Rodríguez Méndez, and Armin Haller. 2020. Towards SHACL Learning from Knowledge Graphs. In *Proceedings of the ISWC 2020 Demos and Industry Tracks (CEUR Workshop Proceedings)*, Vol. 2721. CEUR-WS.org, Globally online, 94–99.

[33] Stefano Ortona, Venkata Vamsikrishna Meduri, and Paolo Papotti. 2018. Robust discovery of positive and negative rules in knowledge bases. In *2018 IEEE 34th International Conference on Data Engineering (ICDE)*. IEEE, 1168–1179.

[34] Harshvardhan J. Pandit, Declan O'Sullivan, and Dave Lewis. 2018. Using Ontology Design Patterns To Define SHACL Shapes. In *Proceedings of the 9th Workshop on Ontology Design and Patterns (CEUR Workshop Proceedings)*, Vol. 2195. CEUR-WS.org, Monterey, USA, 67–71.

[35] Minh-Duc Pham, Linnea Passing, Orri Erling, and Peter Boncz. 2015. Deriving an emergent relational schema from RDF data. In *Proceedings of the 24th International Conference on World Wide Web*. 864–874.

[36] Eric Prud'hommeaux, José Emilio Labra Gayo, and Harold R. Solbrig. 2014. Shape expressions: an RDF validation and transformation language. In *Proceedings of the 10th International Conference on Semantic Systems, SEMANTiCS*. ACM, Leipzig, Germany, 32–40.

[37] QSE-Exact 2022. SPARQL queries used for shape extraction in QSE-Exact (query-based) approach. https://github.com/kashif-rabbani/shacl/blob/main/sparql_queries.txt. Accessed 26th June.

[38] Top Quadrant. 2022. TopBraid. https://www.topquadrant.com/products/topbraid-composer/. Accessed 20th January.

[39] Kashif Rabbani. 2022. QSE GitHub. https://github.com/kashif-rabbani/shacl. Accessed 26th June.

[40] Kashif Rabbani, Matteo Lissandrini, and Katja Hose. 2021. Optimizing SPARQL Queries using Shape Statistics. In *Proceedings of the 24th International Conference on Extending Database Technology, EDBT*. OpenProceedings.org, Nicosia, Cyprus, 505–510.

[41] Kashif Rabbani, Matteo Lissandrini, and Katja Hose. 2022. SHACL and ShEx in the Wild: A Community Survey on Validating Shapes Generation and Adoption. In *Proceedings of the ACM Web Conference 2022*. ACM, Online, Lyon, France. https://www2022.thewebconf.org/PaperFiles/65.pdf

[42] Livio Robaldo. 2021. Towards compliance checking in reified I/O logic via SHACL. In *ICAIL '21: Eighteenth International Conference for Artificial Intelligence and Law, São Paulo Brazil, June 21 - 25, 2021*, Juliano Maranhão and Adam Zachary Wyner (Eds.). ACM, Paulo Brazil, 215–219.

[43] Ognjen Savkovic, Evgeny Kharlamov, and Steffen Lamparter. 2019. Validation of SHACL Constraints over KGs with OWL 2 QL Ontologies via Rewriting. In *The Semantic Web - 16th International Conference, ESWC 2019, Portorož (Lecture Notes in Computer Science)*, Vol. 11503. Springer, Slovenia, 314–329.

[44] Stefan Schmid, Cory Henson, and Tuan Tran. 2019. Using Knowledge Graphs to Search an Enterprise Data Lake. In *The Semantic Web: ESWC 2019 Satellite Events - ESWC (Lecture Notes in Computer Science)*, Vol. 11762. Springer, Portorož, Slovenia, 262–266. https://doi.org/10.1007/978-3-030-32327-1_46

[45] Juan Sequeda and Ora Lassila. 2021. Designing and Building Enterprise Knowledge Graphs. *Synthesis Lectures on Data, Semantics, and Knowledge* 11, 1 (2021), 1–165.

[46] SHACL Shapes 2022. SHACL shapes for DBpedia, LUBM, YAGO-4, and WikiData published on Zenodo. https://doi.org/10.5281/zenodo.5958985. Accessed 26th June.

[47] Umutcan Simsek, Kevin Angele, Elias Kärle, Oleksandra Panasiuk, and Dieter Fensel. 2020. Domain-Specific Customization of Schema.org Based on SHACL. In *ISWC (2) (Lecture Notes in Computer Science)*, Vol. 12507. Springer, Athens, Greece, 585–600.

[48] Blerina Spahiu, Andrea Maurino, and Matteo Palmonari. 2018. Towards Improving the Quality of Knowledge Graphs with Data-driven Ontology Patterns and SHACL. In *Emerging Topics in Semantic Technologies - ISWC 2018 Satellite Events (Studies on the Semantic Web)*, Vol. 36. IOS Press, Satellite, USA, 103–117.

[49] Blerina Spahiu, Riccardo Porrini, Matteo Palmonari, Anisa Rula, and Andrea Maurino. 2016. ABSTAT: Ontology-Driven Linked Data Summaries with Pattern Minimalization. In *ESWC (Satellite Events) (Lecture Notes in Computer Science)*, Vol. 9989. LNCS, Crete, Greece, 381–395.

[50] Thomas Pellissier Tanon, Gerhard Weikum, and Fabian M. Suchanek. 2020. YAGO 4: A Reason-able Knowledge Base. In *The Semantic Web - 17th International Conference, ESWC (Lecture Notes in Computer Science)*, Vol. 12123. Springer, Heraklion, Crete, Greece, 583–596.

[51] Jiao Tao, Evren Sirin, Jie Bao, and Deborah L. McGuinness. 2010. Integrity Constraints in OWL. In *Proceedings of the Twenty-Fourth AAAI Conference on Artificial Intelligence, AAAI*. AAAI Press, Atlanta, Georgia, USA.

[52] Katherine Thornton, Harold Solbrig, Gregory S Stupp, Jose Emilio Labra Gayo, Daniel Mietchen, Eric Prud'Hommeaux, and Andra Waagmeester. 2019. Using shape expressions (ShEx) to share RDF data models and to guide curation with rigorous validation. In *ESWC*. Springer, Cham, 606–620.

[53] Denny Vrandečić and Markus Krötzsch. 2014. Wikidata: a free collaborative knowledgebase. *Commun. ACM* 57, 10 (2014), 78–85.

[54] W3C. 2022. SHACL- core constraint components. https://www.w3.org/TR/shacl/#core-components. Accessed 20th January.

[55] W3C. 2022. W3C: RDF Type. http://www.w3.org/1999/02/22-rdf-syntax-ns#type. Accessed 20th January.

[56] WESO. 2022. RDFShape. http://rdfshape.weso.es. Accessed 20th January.

[57] WikiData-2015 2022. WikiData-2015. https://archive.org/details/wikidata-json-20150518. Accessed 26th June.

[58] Jesse Wright, Sergio José Rodríguez Méndez, Armin Haller, Kerry Taylor, and Pouya Ghiasnezhad Omran. 2020. Schímatos: A SHACL-Based Web-Form Generator for Knowledge Graph Editing. In *ISWC (2) (Lecture Notes in Computer Science)*, Vol. 12507. Springer, Athens, Greece, 65–80.

A APPENDIX

A.1 SHACL Syntax

Figure 1 illustrates an example RDF graph (1a) with its validating shapes (1b). Here, we provide a syntactical example of SHACL shapes for a :Student node shape and :takesCourse property shape. There exist two different types of restrictions in property shapes constraints, namely: *existential restrictions* and *property type* restrictions. The former are defined by *cardinality constraints* using sh:minCount and sh:maxCount properties, while the latter define the target type for the value node of a specific property using sh:class property.

```
sh:StudentShape   a     sh:NodeShape ;
sh:targetClass    :Student
sh:property [ a  sh:PropertyShape ;
                 sh:path     :name ;
                 sh:nodeKind sh:Literal ;
                 sh:datatype xsd:String ;
                 sh:minCount 1 ;
                 sh:maxCount 1 ;   ].
sh:property [ a  sh:PropertyShape ;
                 sh:path     :takesCourse ;
                 sh:nodeKind sh:IRI ;
                 sh:minCount 1 ;
                 sh:class    :Course ;   ].
```

Listing 1: Student Node Shape having two Property Shapes

A.2 QSE-Exact

One of the most common ways to store an RDF graph $\mathcal{G}$ on a file **F** is to represent it as a sequence of triples. Thus, QSE reads **F** line by line and processes it as a stream of $\langle s, p, o\rangle$ triples. The core steps of the QSE approach to extract constraints are described in Algorithm 3. In the first pass, the algorithm parses each $\langle s, p, o\rangle$ triple containing a type declaration and for each entity it stores the set of its entity types and their frequency, i.e., the number of instances for each class (Lines 6-10) in maps Ψ_{ETD} (Entity to Data) and Ψ_{CEC} (Class to Entity Count), respectively. Once the types of all entities and their counts are computed, the algorithm performs a second pass by streaming over **F** again (Lines 11-28) to collect the constraints and the meta-data required to compute support and confidence of each candidate shape. Specifically, it parses the triples to obtain subject and object types of all triples except for the type defined triples as they are already processed in the first pass.

Here, the map Ψ_{ETD}, obtained in the first pass, is used to identify the types of non-literal objects, while the map Ψ_{ETPD} (Entity to Property Data) is updated to contain the candidate property constraints associated with each entity (Lines 13-21).

The constraints' information extracted in the form of maps (Ψ_{ETD}, Ψ_{CEC}) in Algorithm 3 is used as input by the Algorithm 4to compute support and confidence for specific constraints. The algorithm iterates over the map Ψ_{ETD} to extract the map Ψ_{ETPD} mapping entities to candidate property shapes ϕ_s:$\langle \tau_p, \mathsf{T}_p, \mathsf{C}_p\rangle \in \Phi_s$, and retrieves the type of each entity using types information stored in Ψ_{ETD} to build triplets for each distinct triplet of the form $\langle \tau_s, \tau_p, \tau_o\rangle$ (Lines 3-10). The value of support for each distinct triplet is incremented in each iteration and stored in the map Ψ_{Supp}. Additionally, a map Ψ_{PTT} (Property to Types) is populated with properties and their object types for all the properties having max count equal to one (Lines 11-14). Once the support for each distinct triplet is computed, the value of confidence for each triplet is computed using Equation 2 and stored in the map Ψ_{Conf} (Lines 15-16).

We now use the constraints' information extracted in Algorithm 3 and the support/confidence computed in Algorithm 4 to construct the final set of shapes (Algorithm 5). The algorithm iterates over the values of the Ψ_{CTP} map and defines the *shape name* of s, the *shape's target definition* τ_s, and the set of *shape constraints* ϕ_s for each candidate class (Lines 1-3). The map value $\text{Map}\langle\text{Iri}_{\text{Property}}, \text{Set}\rangle$ for each candidate class is used to extract properties and object types to define property constraints P for each *Node Shape* (Lines 4-22). The property constraints specification for P includes sh:path, and depending on sh:nodeKind either sh:class, or sh:datatype. As a matter of fact, the sh:nodeKind constraint can possibly have three types of values: sh:Literal, sh:IRI, and sh:BlankNode. In case of Literal types, the literal object type is used, e.g., xsd:string, xsd:integer, or xsd:date. However, in the case of non-literal object types, the constraint sh:class is used to declare the type of object associated with the candidate property. It is possible to have more than one value for the sh:class and sh:datatype constraints of a candidate property shape, in such cases, we use sh:or constraint to encapsulate multiple values.

Algorithm 3 Extract Constraints (Phase 1 and 2)

Input: Graph $\mathcal{G}$ from File **F**
Output: Ψ_{ETD} , Ψ_{CEC} , Ψ_{CTP}
1: EntityData ⟵ {T: $\text{Set}_{\text{Types}}$, Ψ_{ETPD} = Map⟨Iri, PropertyData⟩}
2: PropertyData ⟵ {T′ : $\text{Set}_{\text{ObjTypes}}$, Count :Int}
3: Ψ_{ETD} = Map⟨Iri, EntityData⟩ ▹ Entity to entity's data
4: Ψ_{CEC} = Map⟨Iri, Int⟩ ▹ Class to entity count
5: Ψ_{CTP} = Map⟨Iri, Map⟨Iri, Set⟩⟩ ▹ Class to props with object types
6: **for** $t \in \mathcal{G}$ ∧ t.p = Type Predicate **do** ▹ ① Entity extraction
7: **if** t.s ∉ Ψ_{ETD} **then**
8: Ψ_{ETD}.insert($t.s$, new EntityData (Set.init($t.o$)))
9: Ψ_{ETD}.insert($t.s$, Ψ_{ETD}.get($t.s$).T.add($t.o$))
10: Ψ_{CEC}.putIfAbsent($t.o$, Set.init(0))
11: Ψ_{CEC}.insert($t.o$, Set.add(Ψ_{CEC}.get($t.o$)+1))
12: **for** $t \in \mathcal{G}$ ∧ t.p != Type Predicate **do** ▹ ② Entity constraints extraction
13: $\lambda_{\text{Type}(t.o)}$ ⟵ ∅ , λ_{Tuple} ⟵ ∅ ▹ Initialize a *type* and *property to type* tuple set
14: **if** $t.o$ is Literal **then**
15: $\lambda_{\text{Type}(t.o)}$.add(getType(t.o))
16: λ_{Tuple}.add(Tuple ⟨ t.p, getType(t.o) ⟩)
17: **else**
18: **for** Iri ∈ Ψ_{ETD}.get($t.o$) **do**
19: $\lambda_{\text{Type}(t.o)}$.add(Iri)
20: λ_{Tuple}.add(Tuple ⟨t.p, Iri⟩)
21: *updatePropertyConstraints*(t.s, λ_{Tuple})
22: **for** Iri ∈ Ψ_{ETD}.get($t.s$.T) **do** ▹ if $t.s \in \Psi_{\text{ETD}}$
23: **if** Ψ_{CTP}.get(Iri) is ∅ **then**
24: Ψ_{CTP}.insert(Iri, ψ.init($t.p$, $\lambda_{\text{Type}(t.o)}$))
25: **if** Ψ_{CTP}.get(Iri).containsKey($t.p$) **then**
26: Ψ_{CTP}.get(Iri).get($t.p$).add($\lambda_{\text{Type}(t.o)}$)
27: **else**Ψ_{CTP}.get(Iri).insert($t.p$, $\lambda_{\text{Type}(t.o)}$)
28: **function** *updatePropertyConstraints (Iri, λ_{Tuple})*
29: entityData = Ψ_{ETD}.get(Iri)
30: **for** tuple t ∈ λ_{Tuple} **do**
31: propertyData = entityData.Ψ_{ETPD}.get($t._1$)
32: **if** propertyData is NULL **then**
33: propertyData = new PropertyData()
34: entityData.Ψ_{ETPD}.insert($t._1$, propertyData)
35: propertyData.T'.add($t._2$) ; propertyData.Count += 1
36: Ψ_{ETD}.insert(Iri, entityData)

Algorithm 4 Compute Support And Confidence (Phase 3)

Input: Ψ_{ETD} , Ψ_{CEC}
Output: Ψ_{Supp} , Ψ_{Conf} , Ψ_{PTT}
Ψ_{Supp} = Map⟨Tuple$_3$, Int⟩ , Ψ_{Conf} = Map⟨Tuple$_3$, Int⟩
Ψ_{PTT} = Map⟨Iri, Set$_{\text{Types}}$⟩ ▷ Property to types having max count one
for ⟨Iri , E_{Data}⟩ ∈ Ψ_{ETD} **do** ▷ Ψ_{ETD} : Map⟨Iri, EntityData⟩
  for τ_s ∈ E_{Data}.get(T: Set$_{\text{Types}}$) **do** ▷ To compute support
    PC: Set$_{Tuple2}$ = constructTupleSet(E_{Data}.get(Ψ_{ETPD}))
    for $T(\tau_p, \tau_o)$ ∈ PC: **do** ▷ PC: Property Constraints Tuple$_2$ Set
      Tuple$_3$ ← $Tuple_3\langle \tau_s, \tau_p, \tau_o \rangle$
      if Ψ_{Supp}.containsKey(Tuple$_3$) **then**
        $v = \Psi_{\text{Supp}}$.get(Tuple$_3$) , Ψ_{Supp}.insert(Tuple$_3$, $v + 1$)
      else Ψ_{Supp}.insert(Tuple$_3$, 1)
  for ⟨Iri , P_{Data}⟩ ∈ E_{Data}.get(Ψ_{ETPD}) **do** ▷ To track max count cardinality
    if P_{Data}.get(Count) = 1 **then**
      Ψ_{PTT}.putIfAbsent(Iri, Set.init())
      Ψ_{PTT}.get(Iri).insert(E_{Data}.get(T: Set$_{\text{Types}}$))
for (Tuple$_3$, $supp$) ∈ Ψ_{Supp} **do**
  $c = \Psi_{\text{CEC}}$.get(Tuple3$_1$) , $conf = \frac{supp}{c}$, Ψ_{Conf}.insert(Tuple$_3$, $conf$)

Algorithm 5 Construct SHACL Shapes (Phase 4)

Input: Ψ_{CTP} Ψ_{Conf}, Ψ_{Supp}, Ψ_{PTT} , ω: min-support, ε: min-confidence, ε': min-confidence for assigning cardinality constraint
Output: $\mathcal{S}\langle s, \tau_s, \Phi_s \rangle$
for (Iri$_{\text{Class}}$, Map⟨Iri$_{\text{Property}}$, Set⟩) ∈ Ψ_{CTP} **do**
  $\Phi_s \leftarrow \emptyset$ ▷ $\Phi_s = \{\phi_{s_1}, \phi_{s_2}, ..., \phi_{s_n}\}$ where ϕ_s:$\langle \tau_{\text{p}}, \text{T}_{\text{p}}, \text{C}_{\text{p}} \rangle$
  s = Iri$_{\text{Class}}$.extractShapeName() , τ_s = Iri$_{\text{Class}}$
  for (p, Set$_{\text{Object Types}}$) ∈ Map⟨Iri$_{\text{Property}}$, Set⟩ **do**
    $\phi_s.\tau_{\text{p}} = p$
    $p.\omega = \Psi_{\text{Supp}}$.get($p$, Set$_{\text{Object Types}}$) ▷ Support of property p
    $p.\varepsilon = \Psi_{\text{Conf}}$.get($p$, Set$_{\text{Object Types}}$) ▷ Confidence of property p
    if $p.\omega > \omega \wedge p.\varepsilon > \varepsilon$ **then**
      if p is Literal **then** *for each* (objType ∈ Set$_{\text{Object Types}}$)
        if |Set$_{\text{Object Types}}$| > 1 **then** encapsulate all $\phi_s.\text{T}_{\text{p}}$ in sh:or
        $\phi_s.\text{T}_{\text{p}}.add($ sh:nodeKind : Literal)
        $\phi_s.\text{T}_{\text{p}}.add($ sh:datatype :objType)
        if Ψ_{PTT}.containsKey(p) ∧ Ψ_{PTT}.get(p).exists(objType) **then**
          $\phi_s.\text{C}_{\text{p}}.add($sh:maxCount : 1)
      else *for each* (objType ∈ Set$_{\text{Object Types}}$)
        if |Set$_{\text{Object Types}}$| > 1 **then** encapsulate all $\phi_s.\text{T}_{\text{p}}$ in sh:or
        $\phi_s.\text{T}_{\text{p}}.add($sh:nodeKind : IRI)
        $\phi_s.\text{T}_{\text{p}}.add($ sh:class :objType)
        if Ψ_{PTT}.containsKey(p) ∧ Ψ_{PTT}.get(p).exists(objType) **then**
          $\phi_s.\text{C}_{\text{p}}.add($sh:maxCount : 1)
    $\phi_s.\text{C}_{\text{p}}.add($sh:minCount : 1) ▷ $if p.\varepsilon > \varepsilon'$
    $\Phi_s.add(\phi_s)$
  $\mathcal{S}.add(s, \tau_s, \Phi_s)$ ▷ if s.$\omega > \omega \wedge \phi_s!\emptyset$

A.3 QSE-Approximate

Here we discuss the QSE-Approximate algorithm in details. The approximate QSE solution has two steps. In the first step, entities of a given graph $\mathcal{G}$ are sampled using NbDRS (Algorithm 6), and in the second step, sampled entities are used to extract shapes using Algorithm 3. Algorithm 6 shows the detailed pseudocode for the first step where graph $\mathcal{G}$ is sampled using NbDRS. It requires graph $\mathcal{G}$ (from a file **F**), sampling percentage (Sampling$_{\text{percentage}}$), and threshold for maximum number of entities to be sampled per reservoir, i.e., τ_{max}. It starts by declaring hash maps (Lines 1-5) required for storing extracted information while parsing each $\langle s, p, o \rangle$ triple t from **F** in the loop starting from Line 7. Triples are filtered based on their type predicate (such as rdf:type and wdt:P31) to extract entities' information (Line 8-40), while non-type triples are filtered on Line 41 to keep count of distinct properties using Ψ_{PC} map.

In NbDRS approach, a reservoir is maintained for each distinct entity type (e.g., reservoir for :Student and :FullProfessor in Figure 1) using a map called Ψ_{SEPC} (Sampled Entities per class). The map Ψ_{RCPC} is designated to store reservoir's capacity for each class and allows to increase its capacity dynamically. Both these maps are initialized (Line 9-10) and if the reservoir for the current type has capacity, it is filled with its entity information using Ψ_{ETD} map (Lines 11-15); otherwise a random number r is generated between zero and the current line number of **F** (Line 17). Here we use the term entity and node interchangeably. The node at index r is called focus node $\widehat{n}$, its left neighbor node $\overleftarrow{n}$ and right neighbor node $\overrightarrow{n}$ are identified and their scope is computed (Lines 19-27). The scope of a node represents the number of types of a specific entity, e.g., a person can have multiple types like Student, Teaching Assistant or a Researcher. The node with a minimum scope is marked as the node to be replaced in the reservoirs (Line 28), it is removed from the reservoirs of its type (Ψ_{SEPC}) and Ψ_{ETD} map (Lines 29-30). The new entity is added at the index of the replaced node (Line 32-33). The real entity count of each type is also computed and stored in Ψ_{CEC} (Class to Entity Count) map. This reservoir sampling

Algorithm 6 NbDRS

Input: Graph $\mathcal{G}$ from File **F**, maximum entity threshold τ_{max}, Sampling$_{\text{percentage}}$
Output: Ψ_{ETD} , Ψ_{CEC} , Ψ_{CTP}
1: Ψ_{ETD} = Map⟨Iri, EntityData⟩ ▷ Entity to entity's data : Algorithm 3 line 1-2
2: Ψ_{SEPC} = Map⟨Iri, List<Iri>⟩ ▷ Sampled entities per class
3: Ψ_{RCPC} = Map⟨Iri, Int⟩ ▷ Reservoir capacity per class
4: Ψ_{CEC} = Map⟨Iri, Int⟩ ▷ Class to entity count
5: Ψ_{PC} = Map⟨Int, Int⟩ ▷ Property count
6: τ_{min} = 1 (minimum entity threshold) ; lineCounter = 0
7: **for** t ∈ $\mathcal{G}$ **do** ▷ t denotes triple $\langle s, p, o \rangle$
8: **if** t.p = Type Predicate **then**
9: Ψ_{SEPC}.putIfAbsent(t.o, new List<>(size: τ_{max}))
10: Ψ_{RCPC}.putIfAbsent(t.o, τ_{min})
11: **if** Ψ_{SEPC}.get (t.o).size() < Ψ_{RCPC}.get (t.o) **then** ▷ Fill the reservoir
12: **if** Ψ_{ETD}.get(t.s).T is $\emptyset$ **then**
13: Ψ_{ETD}.insert(t.s, new EntityData (Set.init(t.o)))
14: Ψ_{ETD}.insert(t.s, Ψ_{ETD}.get(t.s).T.add(t.o))
15: Ψ_{SEPC}.get(t.o).insert(t.s)
16: **else** ▷ Replace random node in the reservoir
17: r = generateRandomNumberBetween(0, lineCounter)
18: **if** $r < \Psi_{\text{SEPC}}$.get(t.o).size() **then**
19: $\overleftarrow{n}$ = NULL ; $\widehat{n}$ = NULL ; $\overrightarrow{n}$.SCOPE = ∞ ; $\overleftarrow{n}$.SCOPE = ∞
20: $\widehat{n} = \Psi_{\text{SEPC}}$.get (t.o).valueAtIndex($r$))
21: **if** r ! = 0 **then** ▷ Avoid first item in reservoir
22: $\overleftarrow{n} = \Psi_{\text{SEPC}}$.get(t.o).valueAtIndex($r - 1$))
23: **if** ∃ $\overleftarrow{n}$ ∈ Ψ_{ETD} **then** $\overleftarrow{n}$.SCOPE = Ψ_{ETD}.get($\overleftarrow{n}$).T.size()
24: **if** r ! = Ψ_{SEPC}.get(t.o).size() −1 **then** ▷ Avoid last item in reservoir
25: $\overrightarrow{n} = \Psi_{\text{SEPC}}$.get(t.o).valueAtIndex($r + 1$))
26: **if** ∃ $\overrightarrow{n}$ ∈ Ψ_{ETD} **then** $\overrightarrow{n}$.SCOPE = Ψ_{ETD}.get($\overrightarrow{n}$).T.size()
27: $\widehat{n}$.SCOPE = Ψ_{ETD}.get($\widehat{n}$).T.size()
28: n = findNodeWithMinimumScope($\overleftarrow{n}, \widehat{n}, \overrightarrow{n}$)
29: ∀ Ψ_{ETD}.T → **if** ∃ t.o ∈ Ψ_{SEPC} **then** Ψ_{SEPC}.get(t.o).removeAtIndex(n)
30: Ψ_{ETD}.remove(n) ; entityData = Ψ_{ETD}.get(t.o)
31: **if** entityData is NULL **then** entityData = new EntityData()
32: entityData.T.add (t.o) ; Ψ_{ETD}.insert(t.s, entityData)
33: Ψ_{SEPC}.get(t.o).add (t.s)
34: Ψ_{CEC}.putIfAbsent(t.o, Set.init(0))
35: Ψ_{CEC}.insert(t.o, Set.add(Ψ_{CEC}.get($t.o$)+1))
36: ▷ Resize reservoir
37: currentRatio = ($\Psi_{\text{SEPC}}.get($t.o$).size()/\Psi_{\text{CEC}}.get($t.o$)) \times 100$
38: newCapacity = Sampling$_{\text{percentage}}$ × Ψ_{SEPC}.get(t.o).size()
39: **if** newCapacity < τ_{max} ∧ currentRatio ≤ Sampling$_{\text{percentage}}$ **then**
40: Ψ_{RCPC}.insert(t.o , newCapacity)
41: **else** Ψ_{PC}.putIfAbsent(t.p, 0) ; Ψ_{PC}.get(t.p).incrementByOne()
42: lineCounter + +

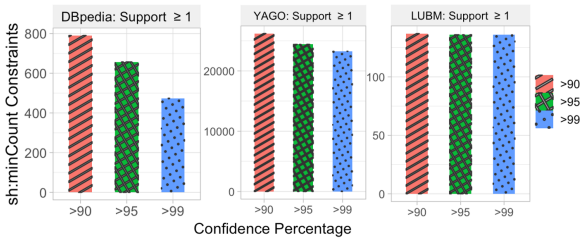


Figure 5: sh:minCount Constraint Analysis

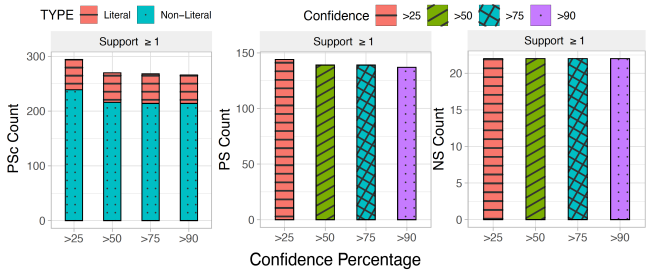


Figure 4: PSc, PS, and NS Pruning on LUBM

approach is called dynamic as it computes the current sampling ratio (with respect to sampled entities and the total entities of the current type/class - Line 37) and new size of the reservoir using sampling percentage and count of sampled entities (Line 38). If the newly computed size is less than τ_{max} (the maximum number of entities to be sampled) and the current sampling ratio is less than the specified sampling percentage, then the reservoir capacity of the current type is increased by the newly computed reservoir's capacity (Lines 39-40).

Once the sampling is performed, the sampled entities of $\mathcal{G}$ are used in the same way as in Algorithm 3 (From line 11) to extract their property information required for shape extraction in the subsequent algorithms.

B EXPERIMENTS

Here we discuss the

B.1 LUBM

Figure 4 shows the result of pruning PSc, PS, and NS for various values of confidence and support $\geq$ 1 on LUBM. As LUBM is a synthetic dataset and only contains 323 PSc (non-literal sh:class constraints), the resulting pruning in the number of PSc and NS is not significant compared to DBpedia and YAGO-4.

B.2 Cardinality Constraints

As discussed in Section 4, QSE makes use of support and confidence computed for PSc to add a min cardinality constraint (sh:minCount) in PS. As a baseline, it assigns min cardinality to PS having a support equivalent to the number of entities of its node shape target class (Table 2). QSE also supports assigning min cardinality constraints based on the user's provided support and confidence. Figure 5 shows the number of min cardinality constraints created for PS on each dataset. Results show that the greater the value for confidence and support, the lower the number of min cardinality constraints assigned to PS.